ANÁLISE DE PROJETO E SISTEMAS · PROJETO FINAL

Trilha Enem

Seu caminho até a aprovação, passo a passo.

Documento de Apoio (AID)
Da Concepção ao Dia 1 do Dev: Blocos 1 a 9 (concepção, requisitos, arquitetura, UML, banco, infraestrutura, planejamento, documentação para o dev e qualidade).

Analista / PO	José Felipe
Arquiteto de Software	Arionaldo Júnior
Engenheiro / DBA	Lucas Leal

UNIPÊ · Centro Universitário de João Pessoa

Data: 06/11/2026 · Versão 1.0

Sumário

0. Como ler este documento 4
0.1 Premissas globais (usadas em todos os blocos) 4
Bloco 1. Concepção e Escopo 5
1.1 Nome, domínio e slogan 5
1.2 Documento de Visão 5
1.3 Atores, papéis e por que cada um é necessário 5
1.4 Personas 6
1.5 Mapa de Stakeholders 6
1.6 Estudo de viabilidade 6
1.7 Riscos iniciais 7
Bloco 2. Requisitos 8
2.1 Requisitos funcionais (34) 8
2.2 Requisitos não funcionais (todos com métrica) 8
2.3 Histórias de usuário (15) 8
Bloco 3. Arquitetura de Software 10
3.1 Registros de Decisão de Arquitetura (ADR) 10
3.2 Stack tecnológica (todas as camadas) 11
3.3 Padrões de design aplicados 11
3.4 Modelo C4 11
Bloco 4. Diagramas UML 14
4.1 Diagrama de Casos de Uso 14
4.2 Diagrama de Classes (15 classes, atributos tipados, visibilidade e relações) 15
4.3 Diagramas de Sequência (3) 16
4.4 Diagrama de Componentes 19
4.5 Diagrama de Deployment 19
4.6 Diagramas de Atividades (2) 21
4.7 Rastreabilidade UML × requisitos 21
Bloco 5. Banco de Dados 23
5.1 Decisão do banco 23
5.2 Modelo Conceitual (ER) 24
5.3 Modelo Lógico 25
5.4 Modelo Físico (DDL executável, PostgreSQL 16) 25
5.5 Dicionário de Dados (entidades com dados pessoais e entidades críticas) 27
5.6 Estratégia de dados 28
Bloco 6. Infraestrutura e DevOps 29
6.1 Diagrama de infraestrutura 29
6.2 Custo mensal de infraestrutura por ambiente 29
6.3 Como a infra garante 99,9% (RNF03) 29
6.4 Pipeline CI/CD (GitHub Actions) 29
6.5 Containerização 30
6.6 Segurança 31
6.7 Observabilidade 31
Bloco 7. Planejamento e Estimativas 32
7.1 WBS (Work Breakdown Structure, 4 níveis) 32
7.2 Estimativa de esforço 32
7.3 Composição da equipe e custo 33
7.4 Matriz de riscos (11 riscos) 33
Bloco 8. Documentação para o Dev 35
8.1 README completo (do clone ao localhost) 35
8.2 Variáveis de ambiente (.env.example) 35
8.3 Contrato de API (OpenAPI 3.0, trecho inicial) 35
8.4 Guia de padrões de código 37
8.5 Git workflow 37
8.6 Sprint 1 detalhado (34 pontos) 37
Bloco 9. Qualidade e Processo 38
9.1 Modelo de domínio e glossário 38
9.2 Estratégia de testes 38

9.3 Metas de cobertura 38
9.4 Definição de Pronto (DoR, Definition of Ready) 38
9.5 Definição de Feito (DoD, Definition of Done) 38
9.6 Metodologia: Scrum, com Kanban na operação 39
Coerência entre os blocos **40**

0. Como ler este documento

O projeto é uma cadeia: cada bloco usa decisões do anterior. Por isso há IDs cruzados (RF, RNF, ADR, P) em todo o texto. Três marcas aparecem nos números:

- **[F]** número vindo de fonte pública consultada (a fonte está citada).
- **[P]** premissa de negócio ou de engenharia da equipe. É uma hipótese, não um dado de mercado, e deve ser defendida na arguição.
- **[COTAR]** valor que precisa ser lido na calculadora oficial (calculator.aws) ou no Glassdoor, com print anexado. Nada foi preenchido sem fonte.

0.1 Premissas globais (usadas em todos os blocos)

ID	Premissa [P]	Por que esse valor
P01	Ano 1: 20.000 alunos cadastrados	Equivale a 0,42% dos 4,81 milhões de inscritos do Enem 2025 [F]. Meta modesta para um produto novo sem grande verba de marketing
P02	2.000 assinantes pagantes (10% dos cadastrados)	Conversão de freemium para pago em plataformas de assinatura costuma ficar em um dígito baixo; 10% é a meta otimista que o plano precisa sustentar. Os cenários de risco usam 5%
P03	Pico esperado: 6.000 simultâneos (30% dos cadastrados no horário de um simulado nacional). Capacidade projetada: 10.000	A margem de 67% cobre crescimento e picos não previstos. O valor também é o exemplo de escalabilidade do enunciado
P04	Planos: Essencial R$ 39,90/mês e Completo R$ 79,90/mês (ticket médio R$ 59,90)	Preço abaixo de cursinhos presenciais, compatível com o público (63% de inscritos isentos [F]). Valor a validar com pesquisa
P05	Cada usuário ativo no pico gera 1 requisição a cada 10 s, ou seja, ≈ 1.000 req/s com 10.000 usuários	Navegação e envio de respostas durante o simulado
P06	Cada tarefa de 1 vCPU / 2 GB atende ≈ 250 req/s sustentados (a validar com teste de carga)	Com cache Redis nas leituras de questões. Resulta em 4 tarefas no pico e 8 como teto
P07	Cada assinante assiste 10 h de vídeo por mês em 720p (≈ 0,7 GB/h), total ≈ 14.000 GB/mês	Base para o custo de transferência de vídeo
P08	Acervo inicial: 300 h de aulas × 1,5 GB/h (3 resoluções) = 450 GB	Base para armazenamento

Bloco 1. Concepção e Escopo

1.1 Nome, domínio e slogan

Nome do sistema: Trilha Enem. **Domínio:** trilhaenem.com.br (verificar disponibilidade em registro.br e busca de marca no INPI antes de registrar).

Por que "Trilha Enem": a funcionalidade central é o plano de estudos personalizado (RF024), isto é, uma trilha que o aluno percorre até a data da prova. O nome descreve o produto sem explicação, e a palavra "Enem" no domínio ajuda o público a encontrar a plataforma em buscas.

Critério	Trilha Enem (escolhido)	Prumo (descartado)	Rota Enem (descartado)
Liga o nome à função principal	Sim: trilha = plano de estudos	Parcial: ideia de direção, sem ligação direta	Parcial: caminho
Busca (palavra "Enem" no domínio)	Sim	Só na variação prumoenem	Sim
Memorização e pronúncia	Fácil	Fácil	Fácil
Risco de nome genérico/em uso	Médio ("trilha" é comum em educação): mitigar com busca no INPI	Baixo	Médio

Decisão: Trilha Enem vence por unir descrição e busca. O risco de nome comum é aceito e tratado na verificação de marca.

Slogan escolhido: "Seu caminho até a aprovação, passo a passo."

Opção	Combina com "Trilha"?	Por quê
Seu caminho até a aprovação, passo a passo. (escolhido)	Sim, muito	"Caminho" e "passo a passo" repetem a metáfora da trilha e descrevem o plano de estudos
Estude com rumo. Chegue com confiança.	Médio	Combinava com "Prumo"; "rumo" não reforça a ideia de trilha
Do primeiro estudo à nota que você sonha.	Pouco	Promete resultado (nota), difícil de sustentar

1.2 Documento de Visão

Problema real. O Enem é a principal porta de entrada para o ensino superior. A edição de 2025 teve 4.811.338 inscritos confirmados, 11,22% a mais que em 2024 [F: balanço do Inep/MEC de 23/07/2025, divulgado por SBT News e CNN Brasil]. Dois fatos desse balanço definem o problema:

- 63% dos inscritos eram isentos da taxa (3.049.710 isentos contra 1.761.628 pagantes) [F]. A maioria vem de escola pública e não tem acesso a cursinho caro nem a correção individual de redação.
- A faixa etária mais numerosa tem 17 anos (1.081.120 inscritos, 22,5%) [F]. O público central é de adolescentes, o que traz exigências de proteção de dados (LGPD) e de consentimento dos responsáveis.

Além disso, cerca de 70% dos inscritos fizeram as duas provas (abstenção de 27% no primeiro dia) [F: O Povo, 16/11/2025]. Ou seja, muitos desistem no caminho, e a falta de rotina e de feedback contribui para isso. Para 2026, os números oficiais devem ser atualizados no Painel Enem 2026 do Inep antes da entrega.

Os dois problemas que a plataforma ataca: (1) falta de direção (muito conteúdo, nenhuma priorização até a prova) e (2) falta de feedback individual, principalmente na redação.

Proposta de valor. Em uma única plataforma o aluno tem aulas, simulados com diagnóstico por área, correção de redação por rubrica e mentoria, tudo organizado em um plano de estudos que se ajusta ao desempenho e à data da prova, por um preço acessível.

Público-alvo: estudantes do ensino médio e egressos (foco em 16 a 19 anos), responsáveis legais (que autorizam e pagam), professores parceiros e a equipe administrativa.

Metas mensuráveis do ano 1 (derivadas das premissas P01 a P04):

Meta	Valor	Origem
Alunos cadastrados	20.000 (0,42% dos inscritos de 2025)	P01
Assinantes pagantes	2.000 (0,11% dos 1,76 milhão de inscritos pagantes de 2025)	P02
Receita bruta mensal esperada com 2.000 assinantes	R$ 119.800 (2.000 × R$ 59,90)	P02 × P04
Correção de redação	≤ 72 h em 95% dos casos	RNF11
Disponibilidade	99,9% por mês	RNF03

Fora do escopo (cada item reduz arquitetura e custo):

- Aplicativo mobile nativo (versão web responsiva): decisão que evita duas bases de código com equipe de 5 desenvolvedores.
- Aulas ao vivo: exigiriam infraestrutura de streaming em tempo real, muito mais cara que vídeo sob demanda.
- Correção de redação por IA: risco pedagógico e de confiança, e fora do prazo do MVP.
- Integração com SISU ou inscrição oficial no Enem: dependência de sistemas do governo.
- Marketplace aberto de professores: somente professores curados pelo administrador, para controlar qualidade.

Por que declarar o fora do escopo: escopo vago gera arquitetura vaga. Esta lista é o que justifica monolito, uma região da nuvem e 8 sprints.

1.3 Atores, papéis e por que cada um é necessário

Ator	Papel no sistema	Por que é necessário	O que acontece sem ele
Aluno	Estuda, faz simulados, envia redação, agenda mentoria, assina plano	É o usuário final e a fonte de receita. Os requisitos de carga e de LGPD derivam dele (maioria com 17 anos)	Não há produto
Professor	Publica aulas e questões, corrige redações, atende mentorias	O diferencial do produto é feedback humano individual. Só o professor produz conteúdo qualificado e corrige com rubrica	Plataforma vira repositório de vídeos sem feedback
Administrador	Gerencia usuários, turmas, planos, preços e permissões; acompanha o painel de gestão	Controla qualidade e acesso. É quem concede o papel de Professor, evitando um marketplace aberto (fora do escopo)	Sem curadoria nem governança de acesso
Financeiro/Suporte	Concilia pagamentos, trata estornos e atende	Dinheiro e atendimento exigem rotina própria e rastreável. Juntar os dois	Inadimplência e dúvidas

	chamados	mantém o mínimo de 4 atores sem inflar a documentação	ficam sem dono
Gateway de pagamento (externo)	Processa PIX e cartão e avisa por webhook	Evita construir e certificar processamento de pagamento	Sem receita
Serviço de e-mail (externo)	Envia confirmações, lembretes e recuperações	Confirmação de consentimento e recuperação de senha dependem dele	Cadastro de menores fica bloqueado

Os dois atores externos aparecem no diagrama de casos de uso (Bloco 4) e no C4 de contexto (Bloco 3).

1.4 Personas

	Ana Beatriz Souza, 17 anos (Aluna)	Prof. Marcos Oliveira, 34 anos (Professor)	Carla Mendes, 29 anos (Administradora e Financeiro)	Lúcia Souza, 44 anos (Responsável)
Contexto	3º ano em escola estadual de João Pessoa. Estuda à noite, pelo celular, com franquia de dados limitada	Professor de Redação, já atuou em cursinho, quer renda complementar e escala	Gestora financeira e operacional da plataforma	Mãe de Ana, paga a assinatura, usa o celular para tudo
Motivação real	Passar em Engenharia Civil na UFPB. Tirou 580 na redação do último simulado e não sabe o que melhorar	Corrigir com rubrica padronizada (5 competências) sem planilhas e com fila organizada	Fechar o mês sem divergência entre pagamentos e acessos liberados	Saber que os dados da filha estão protegidos e poder cancelar
Requisitos que ela gera	RF014 (vídeo adaptativo 360p), RF018, RF024, RNF07	RF022, RF023	RF012, RF030	RF002, RF006, RF010

As personas se baseiam em fatos: faixa etária de 17 anos, 63% de isentos [F] e correção de redação como maior dor. Os nomes são fictícios.

1.5 Mapa de Stakeholders

Stakeholder	Interesse	Influência	Estratégia
Alunos	Passar na prova, preço baixo	Alta (pagam e geram boca a boca)	Gerenciar de perto: pesquisa e feedback a cada sprint
Responsáveis legais	Segurança, transparência, autorizar	Alta (sem consentimento menor não usa)	Fluxo de consentimento claro (RF002)
Professores	Renda, ferramenta de correção simples	Alta (sem eles não há conteúdo)	Envolver na definição da rubrica e da fila
Equipe e sócios da Trilha Enem	Retorno e prazo	Alta	Reuniões de revisão por sprint
Inep/MEC	Regras e matriz do exame	Alta sobre o conteúdo (mudança de matriz ou datas muda o produto)	Monitorar editais; manter conteúdo atualizável
ANPD (LGPD)	Proteção de dados	Alta (sanções)	Conformidade desde o desenho (RNF06)
Gateway de pagamento	Volume de transações	Média	Contrato revisado e plano B de gateway
Provedor de nuvem (AWS)	Consumo	Média	Alertas de custo
Concorrentes (cursinhos e plataformas)	Participação de mercado	Média	Diferencial em feedback individual e preço
Escolas parceiras (futuro)	Desempenho dos alunos	Baixa no MVP	Fora do escopo; registrar no backlog

1.6 Estudo de viabilidade

Técnica.

Ponto	Avaliação	Justificativa e risco
Vídeo sob demanda	Viável	Armazenamento + CDN com URLs assinadas. Alternativa descartada: servir vídeo pela aplicação, que saturaria as tarefas no pico (ADR-008)
Simulado com 10.000 simultâneos	Viável com teste de carga	Respostas enviadas em lote a cada 30 s (RNF12) reduzem a escrita de 1.000 para ≈ 333 req/s. Cache de questões em Redis
Pagamentos PIX e cartão	Viável	Usar gateway em vez de construir (ADR-009)
LGPD com menores	Viável, exige atenção	Consentimento do responsável para menores de 18 (RF002) e PII marcada no dicionário. Confirmar interpretação do art. 14 da LGPD com orientação jurídica
Prazo de 8 sprints	Viável	293 pontos estimados contra 320 de capacidade (Bloco 7)

Operacional.

Ponto	Avaliação	Cálculo ou observação
Capacidade de correção	Ponto crítico	600 assinantes Completo (30% dos 2.000) × 2 redações/mês = 1.200 redações × 20 min = 400 h/mês. Com 80 h/mês por professor corretor: 5 professores. Valor pago por redação: [COTAR] com a equipe
Produção de conteúdo	Ponto crítico	300 h de aulas no lançamento. Definir quem grava e revisa
Sazonalidade	Risco	Demanda concentrada antes de novembro. Infra com autoscaling (2 a 8 tarefas) e custo calculado em dois cenários
Suporte	Viável	Horário comercial com SLA de 1ª resposta em 24 h (RF029)
Modelo de receita	Assinatura	Descartado: gratuito com anúncios, pois não paga a correção humana

Custo de infraestrutura e de equipe: ver Bloco 6 e Bloco 7. O custo mensal usa preços de nuvem consultados e deixa explícitas as linhas que ainda exigem a calculadora oficial.

1.7 Riscos iniciais

Escala: probabilidade (P) e impacto (I) de 1 a 5; severidade = P × I.

ID	Risco	P	I	Sev.	Mitigação
R01	Pico de acesso derruba o simulado	3	5	15	Autoscaling, cache, lote de 30 s, teste de carga antes do Enem
R02	Vazamento de dados de menores	2	5	10	Criptografia, RBAC, auditoria, PII mínima (RNF05 e RNF06)
R03	Conversão abaixo de 10% (P02)	4	4	16	Plano gratuito com limite, cenário de 5% no financeiro
R04	Falta de professores corretores	3	4	12	Limite de redações por plano (RF023), fila com prazo
R05	Custo de transferência de vídeo acima do previsto	3	4	12	Qualidade adaptativa, métricas de consumo, alertas de custo
R06	Atraso no desenvolvimento	3	3	9	Escopo Must primeiro, buffer de 9% na capacidade

Bloco 2. Requisitos

2.1 Requisitos funcionais (34)

Prioridade MoSCoW: **Must** (sem isso o sistema não cumpre o problema), **Should** (importante, mas há contorno), **Could** (desejável), **Won't** (fora desta versão).

ID	Requisito	Ator	MoSCoW	Justificativa
RF001	Cadastrar aluno (nome, e-mail, data de nascimento, senha)	Aluno	Must	Porta de entrada. Sem cadastro não há plano, histórico nem pagamento
RF002	Exigir e registrar consentimento do responsável legal para menores de 18 anos	Aluno/Responsável	Must	A maior faixa etária tem 17 anos [F]. LGPD art. 14. Adotamos o corte de 18 por prudência jurídica
RF003	Login e logout com token de acesso e renovação	Todos	Must	Base da segurança (RNF05)
RF004	Recuperar senha por e-mail com link de uso único	Todos	Must	Evita chamados de suporte
RF005	Perfis e permissões (Aluno, Professor, Administrador, Suporte/Financeiro)	Administrador	Must	Controle de acesso (OWASP A01)
RF006	Exportar e excluir dados do titular	Aluno/Responsável	Must	Direitos do titular na LGPD (RNF06)
RF007	Listar planos e preços	Aluno	Must	Pré-requisito da assinatura
RF008	Assinar plano com PIX	Aluno	Must	Meio de pagamento mais usado no Brasil e item da arguição
RF009	Assinar com cartão e cobrança recorrente	Aluno	Should	Reduz inadimplência; PIX cobre o essencial
RF010	Cancelar assinatura, com arrependimento em 7 dias	Aluno	Must	Código de Defesa do Consumidor art. 49 (compra online)
RF011	Emitir recibo e histórico de pagamentos	Aluno/Financeiro	Should	Transparência e menos chamados
RF012	Conciliar pagamentos, estornos e chargebacks via webhook	Financeiro	Must	Persona Carla: acesso liberado deve bater com dinheiro recebido
RF013	Publicar, editar e despublicar aulas (vídeo e PDF)	Professor	Must	Conteúdo é o núcleo do produto
RF014	Navegar por área/competência e assistir com qualidade adaptativa	Aluno	Must	Persona Ana: dados móveis limitados; mínimo de 360p
RF015	Registrar progresso e retomar de onde parou	Aluno	Must	Alimenta o plano de estudos (RF024)
RF016	Cadastrar questões (enunciado, alternativas, gabarito, área, habilidade, dificuldade)	Professor	Must	Matéria-prima dos simulados
RF017	Gerar simulado por área, tempo e dificuldade	Aluno	Must	Treino do formato da prova
RF018	Realizar simulado cronometrado com salvamento automático a cada 30 s	Aluno	Must	Queda de conexão não pode apagar respostas (RNF12)
RF019	Corrigir automaticamente e calcular nota estimada	Sistema	Must	Feedback imediato. A nota é estimativa por acertos ponderados por dificuldade, e não afirma ser TRI
RF020	Relatório de desempenho por área e habilidade	Aluno	Must	Transforma resultado em direção de estudo
RF021	Enviar redação (texto digitado, com tema)	Aluno	Must	Maior dor do público (Bloco 1)
RF022	Corrigir redação nas 5 competências (0 a 200 cada, total 0 a 1.000) com comentários	Professor	Must	Replica a rubrica oficial do Enem
RF023	Fila de correção com prazo de 72 h e limite mensal por plano	Administrador/Professor	Should	Controla a capacidade de 400 h/mês (Bloco 1.6)
RF024	Gerar plano de estudos até a data da prova com base no desempenho	Aluno	Must	Diferencial do produto e origem do nome "Trilha"
RF025	Agendar mentoria nos horários do professor	Aluno/Professor	Should	Suporte humano; produto funciona sem ela
RF026	Enviar notificações por e-mail (lembrete, redação corrigida, pagamento)	Sistema	Should	Retenção; assíncrono (ADR-011)
RF027	Gerenciar usuários, turmas e planos	Administrador	Must	Governança
RF028	Abrir chamado de suporte com categoria e anexo	Aluno	Must	Canal único de atendimento
RF029	Atender, priorizar e encerrar chamados com SLA de 1ª resposta em 24 h	Suporte	Must	Compromisso mensurável
RF030	Painel de gestão: receita, inadimplência, simulados por dia, usuários ativos	Administrador/Financeiro	Should	Decisão baseada em dados
RF031	Ranking opcional entre alunos	Aluno	Could	Engajamento, mas traz questões de privacidade
RF032	Aplicativo mobile nativo	Aluno	Won't	Fora do escopo (Bloco 1.2)
RF033	Aulas ao vivo	Aluno/Professor	Won't	Infra de streaming em tempo real é cara
RF034	Correção de redação por IA	Professor	Won't	Risco pedagógico e de confiança

Mapeamento com as 12 funcionalidades macro: F1 cadastro e autenticação = RF001 a RF006; F2 assinatura = RF007 a RF011; F3 aulas = RF013 a RF015; F4 banco de questões = RF016; F5 simulados = RF017 a RF019; F6 desempenho = RF020; F7 redação = RF021 a RF023; F8 plano de estudos = RF024; F9 mentoria = RF025; F10 pagamentos = RF008, RF009, RF012; F11 gestão = RF027; F12 suporte e dashboards = RF028 a RF030.

2.2 Requisitos não funcionais (todos com métrica)

ID	Categoria	Requisito mensurável	Prior.	Por que essa métrica e não outra
RNF01	Desempenho	p95 ≤ 300 ms em leituras e ≤ 500 ms em escritas, medido no load balancer, com 10.000 usuários simultâneos	Must	É a métrica do enunciado; p95 representa a experiência da maioria sem esconder lentidão como a média faria
RNF02	Escalabilidade	Suportar 10.000 usuários simultâneos (esperado 6.000) durante 2 h, validado por teste de carga	Must	P03. Simulado nacional gera o pico

RNF03	Disponibilidade	99,9% por mês (≤ 43,8 min de indisponibilidade)	Must	Falha durante um simulado destrói a confiança. A infra composta chega a ≈ 99,93% (Bloco 6)
RNF04	Recuperação	RPO ≤ 5 min, RTO ≤ 30 min (restauração) e failover de zona ≤ 5 min	Must	PITR do banco entrega RPO de minutos; o RTO acomoda a restauração completa
RNF05	Segurança	TLS 1.2+; senhas com bcrypt custo ≥ 12; JWT de acesso de 15 min e renovação de 7 dias; 5 tentativas de login/min por IP; 0 vulnerabilidades críticas ou altas abertas no deploy	Must	Reduz risco de credenciais vazadas e de componentes vulneráveis
RNF06	LGPD	100% dos campos PII marcados no dicionário; consentimento registrado (data e versão) para 100% dos menores; resposta ao titular em ≤ 15 dias	Must	Público majoritariamente menor; LGPD art. 19 fixa 15 dias para declaração completa
RNF07	Usabilidade móvel	LCP ≤ 2,5 s em 4G; layout a partir de 360 px; vídeo com opção de 360p	Must	Persona Ana usa celular com dados limitados; 2,5 s é o limite “bom” do Core Web Vitals
RNF08	Acessibilidade	WCAG 2.1 nível AA; aulas com legendas	Should	Inclusão; Enem tem 116.541 atendimentos especializados aprovados em 2025 [F]
RNF09	Manutenibilidade	Cobertura de testes ≥ 80% no domínio e ≥ 70% global; complexidade ciclomática ≤ 10 por método	Must	Dev novo precisa de segurança para alterar código
RNF10	Observabilidade	100% das requisições com correlation-id; alertas disparam em ≤ 5 min	Must	Detecta degradação antes do aluno reclamar
RNF11	Prazo de correção	Redação corrigida em ≤ 72 h em 95% dos casos	Should	Prazo que mantém o aluno engajado e é viável com 5 professores
RNF12	Integridade do simulado	Perda máxima de 30 s de respostas em queda de conexão; reenvio idempotente	Must	Lote de 30 s reduz a carga (P05) e define a tolerância máxima

2.3 Histórias de usuário (15)

Formato: *Como [persona], quero [ação], para [objetivo].* Critérios em Given/When/Then. Pontos em escala de Fibonacci.

- **US01 (8 SP) Cadastro com consentimento.** Como Ana, quero me cadastrar, para começar a estudar. *Given* que informo data de nascimento de menor de 18 anos, *when* envio o formulário, *then* o sistema pede o e-mail do responsável e só ativa a conta após o consentimento dele. (RF001, RF002)
- **US02 (3 SP) Login seguro.** Como Ana, quero entrar com e-mail e senha, para acessar meus estudos. *Given* credenciais corretas, *when* faço login, *then* recebo acesso por 15 minutos e o sistema renova a sessão. *Given* 5 tentativas erradas em 1 minuto, *then* o IP é bloqueado temporariamente. (RF003)
- **US03 (8 SP) Assinar por PIX.** Como Lúcia, quero pagar o plano da minha filha por PIX, para liberar o acesso. *Given* um plano escolhido, *when* gero o PIX e o gateway confirma, *then* a assinatura fica ativa em até 1 minuto. (RF008)
- **US04 (5 SP) Cancelar em 7 dias.** Como Lúcia, quero cancelar logo após comprar, para ter o dinheiro de volta. *Given* assinatura com menos de 7 dias, *when* peço cancelamento, *then* o acesso é encerrado e o estorno é solicitado. (RF010)
- **US05 (5 SP) Assistir e retomar aula.** Como Ana, quero retomar a aula de onde parei, para não perder tempo. *Given* aula pausada aos 12 min, *when* volto à aula, *then* ela inicia aos 12 min. (RF014, RF015)
- **US06 (5 SP) Gerar simulado.** Como Ana, quero montar um simulado por área e tempo, para treinar o ritmo. *Given* escolho Matemática e 90 min, *when* gero o simulado, *then* recebo questões inéditas para mim naquela área. (RF017)
- **US07 (13 SP) Fazer simulado com segurança.** Como Ana, quero que minhas respostas sejam salvas sozinhas, para não perder nada se a internet cair. *Given* simulado em andamento, *when* a conexão cai por 20 s, *then* ao voltar perco no máximo as respostas dos últimos 30 s. (RF018, RNF12)
- **US08 (8 SP) Ver desempenho por área.** Como Ana, quero ver onde errei mais, para saber o que estudar. *Given* simulado concluído, *when* abro o relatório, *then* vejo acertos por área e habilidade. (RF019, RF020)
- **US09 (5 SP) Enviar redação.** Como Ana, quero enviar minha redação, para receber correção. *Given* plano Completo com saldo, *when* envio o texto, *then* ele entra na fila e o saldo diminui em 1. (RF021)
- **US10 (8 SP) Corrigir por competência.** Como Marcos, quero dar nota em cada uma das 5 competências, para o aluno saber o que melhorar. *Given* redação na fila, *when* registro notas de 0 a 200 e comentários, *then* a nota total é calculada e o aluno é notificado. (RF022)
- **US11 (13 SP) Plano de estudos.** Como Ana, quero um cronograma até a prova, para estudar o que mais preciso. *Given* data da prova e resultado de um simulado, *when* gero o plano, *then* recebo metas semanais ordenadas pelas áreas mais fracas. (RF024)
- **US12 (5 SP) Agendar mentoria.** Como Ana, quero reservar um horário com o professor, para tirar dúvidas. *Given* horários livres do Marcos, *when* reservo um, *then* ambos recebem confirmação e o horário some da lista. (RF025)
- **US13 (8 SP) Conciliar pagamentos.** Como Carla, quero ver divergências entre pagamento e acesso, para corrigi-las. *Given* webhook de estorno, *when* o sistema o processa, *then* o acesso do aluno é suspenso e o fato aparece no painel. (RF012)
- **US14 (5 SP) Abrir e atender chamado.** Como Ana, quero abrir um chamado com anexo, para resolver um problema. *Given* chamado aberto, *when* o suporte responde em até 24 h, *then* Ana recebe aviso por e-mail. (RF028, RF029)
- **US15 (5 SP) Excluir dados da filha.** Como Lúcia, quero exportar e excluir os dados da minha filha, para exercer meus direitos. *Given* solicitação do responsável, *when* confirmo a exclusão, *then* os dados pessoais são removidos ou anonimizados em até 15 dias. (RF006)

Bloco 3. Arquitetura de Software

Regra deste bloco: toda decisão tem um ADR (contexto, decisão, alternativas descartadas, consequências), e o motivo sempre cita um requisito do Bloco 2 ou uma premissa do item 0.1.

3.1 Registros de Decisão de Arquitetura (ADR)

ADR-001: Monolito modular em vez de microserviços

- **Contexto:** equipe de 5 desenvolvedores, 8 sprints, 12 funcionalidades, RNF02 (10.000 usuários) e P05/P06 (≈ 1.000 req/s, 4 a 8 tarefas).
- **Decisão:** uma aplicação Spring Boot dividida em módulos com fronteiras explícitas (Spring Modulith): Identidade, Assinatura, Conteúdo, Avaliação, Redação, Plano de Estudos, Suporte e Relatórios.
- **Descartadas:** (a) microserviços, que exigiriam um banco por serviço, tracing distribuído, pipeline por serviço e rede entre serviços, sem ganho, porque 1.000 req/s se atendem escalando a mesma aplicação horizontalmente; (b) monolito sem módulos, por acoplamento e risco de virar "big ball of mud".
- **Consequências:** um único PostgreSQL (coerente com o Bloco 5); escala horizontal por tarefas sem estado; se o módulo de Avaliação passar de 70% da carga em produção, ele é candidato a extração (gatilho documentado).

ADR-002: Java 21 + Spring Boot 3 no backend

- **Contexto:** assinatura, pagamento e simulado exigem transações consistentes (RF008, RF012, RF018); RNF05 exige segurança madura.
- **Decisão:** Java 21 (LTS) com Spring Boot 3, Spring Security e Spring Data JPA.
- **Descartadas:** Node.js/NestJS (tipagem opcional e transações de domínio mais manuais) e Python/FastAPI (menor vazão por instância para o pico de P06).
- **Consequências:** Virtual Threads do Java 21 ajudam em I/O concorrente. Fator secundário, tratado como risco de cronograma (R06) e não como motivo técnico: a equipe já tem prática na stack, o que reduz o risco de prazo de 8 sprints.

ADR-003: Next.js (React + TypeScript) com SSR/SSG no frontend

- **Contexto:** páginas públicas (landing, planos, conteúdo sobre o Enem) precisam ser indexáveis e rápidas (RNF07: LCP ≤ 2,5 s em 4G).
- **Decisão:** Next.js, com renderização no servidor nas páginas públicas e renderização no cliente na área logada.
- **Descartada:** SPA React puro, pior para busca e para o LCP.
- **Consequências:** mais um contêiner (ou o mesmo domínio via CDN) a operar.

ADR-004: PostgreSQL em vez de MongoDB ou MySQL

- **Contexto:** dados fortemente relacionais (assinatura, pagamento, acesso, tentativas, respostas), relatórios agregados (RF020, RF030) e LGPD com campos controlados.
- **Decisão:** PostgreSQL 16 gerenciado (RDS).
- **Descartadas:** MongoDB, porque integridade entre assinatura, pagamento e acesso exigiria validação no código, sem chaves estrangeiras; MySQL, por oferecer menos recursos úteis aqui (JSONB, índices parciais, window functions e CTEs mais completas para relatórios).
- **Consequências:** modelo relacional normalizado até a 3FN (Bloco 5); JSONB só para a rubrica de correção.

ADR-005: REST/JSON com contrato OpenAPI 3.0

- **Contexto:** clientes são navegadores; o Bloco 8 exige OpenAPI.
- **Decisão:** API REST versionada em /api/v1.
- **Descartadas:** gRPC (sem serviços internos que justifiquem; exigiria proxy para navegador) e GraphQL (complexidade de cache e de autorização por campo).
- **Consequências:** cache HTTP/CDN possível em leituras públicas.

ADR-006: AWS em sa-east-1 com ECS Fargate

- **Contexto:** RNF01 (p95 ≤ 300 ms), RNF03 (99,9%), sem equipe de operações dedicada.
- **Decisão:** ECS Fargate atrás de um Application Load Balancer, na região de São Paulo.
- **Descartadas:** EKS (só o plano de controle custa US$ 0,10/h, cerca de US$ 73/mês antes de qualquer nó [F: tabela comparativa de preços publicada pela Oracle]), Lambda (início a frio da JVM e conexões ao RDS ameaçam o p95) e EC2 puro (patching e gestão de nós).
- **Região:** sa-east-1 em vez de us-east-1, para manter a latência baixa para usuários no Brasil. O custo é maior: RDS db.t4g.medium custa US$ 0,137/h em São Paulo contra US$ 0,065/h na Virgínia [F: Spare Cores], e o Fargate chega a ser 72% mais caro [F: Oracle]. Aceitamos o custo para respeitar RNF01 e RNF07.

ADR-007: Cache com Redis (ElastiCache)

- **Contexto:** leitura intensiva de questões e catálogos compartilhada por 2 a 8 tarefas; o banco é uma instância db.t4g.medium.
- **Decisão:** Redis para catálogo, questões publicadas, rate limit e lista de revogação de refresh tokens.
- **Descartadas:** cache em memória da aplicação (inconsistente entre tarefas) e nenhum cache (sobrecarrega o banco).
- **Consequências:** o Redis não é fonte de verdade. Se cair, o sistema degrada para o banco com latência maior.

ADR-008: Vídeo em S3 + CloudFront com URLs assinadas e HLS adaptativo

- **Contexto:** RF014 (acesso só para assinantes, qualidade adaptativa), persona Ana, P07 (≈ 14.000 GB/mês).
- **Decisão:** arquivos em S3, entrega por CloudFront com URL assinada de curta duração.
- **Descartadas:** YouTube não listado (não controla acesso por assinatura) e servir pelo backend (saturaria as tarefas no pico).
- **Consequências:** a transferência de vídeo é a linha de custo mais incerta (R05).

ADR-009: Gateway de pagamento terceirizado, camada Adapter

- **Contexto:** RF008, RF009 e RF012 exigem PIX, cartão recorrente e webhook.
- **Decisão (provisória):** Mercado Pago, que oferece PIX e assinaturas recorrentes em uma API com SDK Java; acesso por um Adapter, para trocar de gateway sem mexer no domínio.

- **Descartadas:** processar pagamento direto (escopo de certificação PCI-DSS) e Stripe como primário (suporte a PIX e assinaturas em reais a validar).
- **Consequências:** taxas e contrato precisam ser validados [COTAR] antes de fechar.

ADR-010: Autenticação com JWT sem estado

- **Contexto:** tarefas escalam de 2 a 8 sem sessão compartilhada (ADR-001, ADR-006).
- **Decisão:** token de acesso de 15 min e refresh rotativo de 7 dias, com revogação em Redis.
- **Descartada:** sessão em servidor (exigiria armazenamento compartilhado de sessões).
- **Consequências:** revogação imediata é mais difícil, mitigada pelo TTL curto.

ADR-011: Processamento assíncrono com SQS

- **Contexto:** e-mails, notificações e conciliação não podem atrasar a resposta (RNF01).
- **Decisão:** Amazon SQS, com consumidores idempotentes dentro do monolito.
- **Descartadas:** processar na requisição (aumenta o p95) e Kafka (operação desproporcional para o volume).
- **Consequências:** mensagens podem chegar em duplicidade, por isso os consumidores são idempotentes.

3.2 Stack tecnológica (todas as camadas)

Camada	Escolha	Descartada	Motivo técnico
Frontend	Next.js + TypeScript	SPA React (Vite)	SSR para busca e LCP (RNF07, ADR-003)
Backend	Java 21 + Spring Boot 3	Node/NestJS, Python/FastAPI	Transações ACID e segurança madura (ADR-002)
API	REST/JSON + OpenAPI 3.0	gRPC, GraphQL	Clientes web, cache HTTP, contrato exigido (ADR-005)
Banco	PostgreSQL 16 (RDS Multi-AZ)	MongoDB, MySQL	Integridade relacional e relatórios (ADR-004)
Cache	Redis (ElastiCache)	Cache local, sem cache	Compartilhado entre tarefas (ADR-007)
Fila	Amazon SQS	Kafka, síncrono	Gerenciado e barato (ADR-011)
Vídeo e arquivos	S3 + CloudFront	YouTube, backend	Acesso controlado e escala (ADR-008)
Autenticação	JWT + refresh rotativo	Sessão em servidor	Escala sem estado (ADR-010)
Infra	AWS ECS Fargate, sa-east-1	EKS, Lambda, EC2	Sem operação de nós, p95 estável (ADR-006)
IaC	Terraform	Console manual	Ambientes reproduzíveis (dev, staging, prod iguais)
CI/CD	GitHub Actions	Jenkins	Sem servidor para manter; integra com o repositório
E-mail	Amazon SES	SendGrid	Mesma nuvem e menor custo por e-mail [COTAR]
Observabilidade	CloudWatch + Micrometer	Datadog	Sem custo por host adicional
Testes	JUnit 5, Testcontainers, Playwright	Testes manuais	Automação exigida pela DoD (Bloco 9)

3.3 Padrões de design aplicados

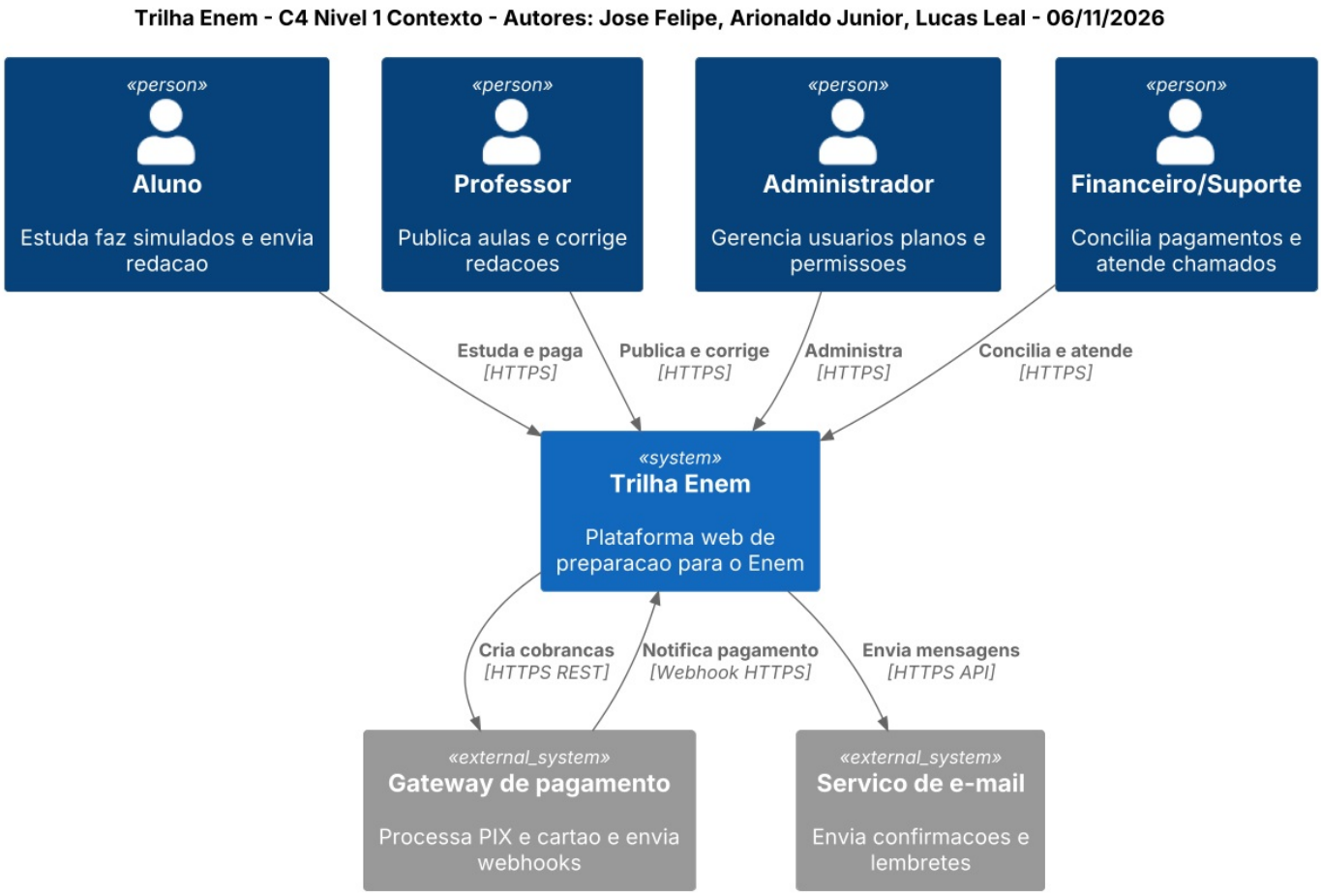
Padrão	Onde	Por que
Repository (Spring Data)	Acesso a dados de todos os módulos	Isola o domínio do SQL e permite testes com banco em contêiner
Adapter	Gateway de pagamento e e-mail	Trocar o fornecedor sem alterar o domínio (ADR-009)
Strategy	Cálculo da nota estimada e geração do plano de estudos	Algoritmos trocáveis (acertos ponderados hoje, TRI no futuro) sem alterar o serviço
Factory	Montagem do Simulado (por área, tempo e dificuldade)	Encapsula a regra de seleção de questões
State	Status de Assinatura (PENDENTE, ATIVA, CANCELADA) e de Redação (ENVIADA, EM_CORREÇÃO, CORRIGIDA)	Evita condicionais espalhadas e garante transições válidas
Domain Events	Pagamento confirmado libera acesso e dispara e-mail	Desacopla módulos dentro do monolito

Padrões considerados e descartados: **Saga** (só faz sentido com transações entre serviços; aqui a transação é local ao banco único) e **CQRS completo** (basta usar views materializadas para relatórios; duplicar modelos de escrita e leitura custaria mais que entrega).

3.4 Modelo C4

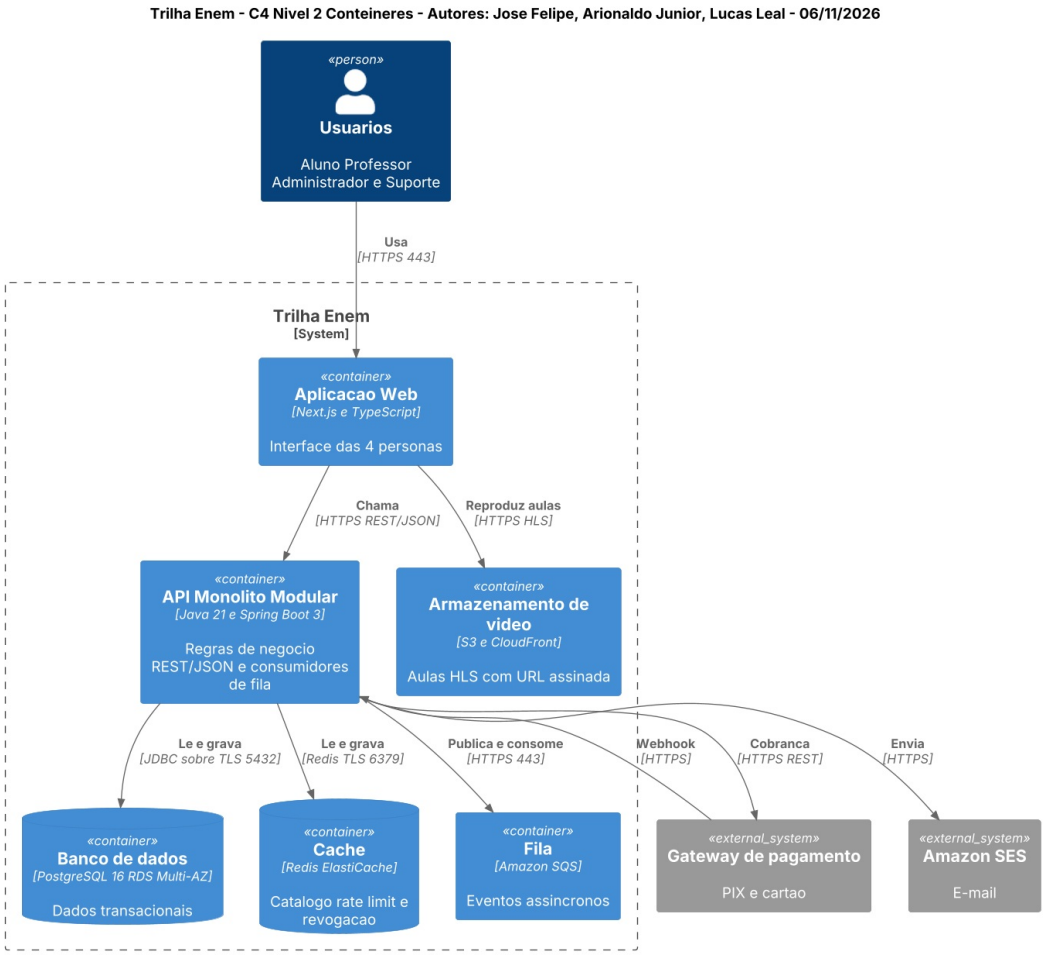
Os quatro níveis abaixo foram renderizados a partir do código PlantUML (fontes na pasta `diagramas/` do pacote de entrega). Todos trazem título, autores e data.

Nível 1: Contexto



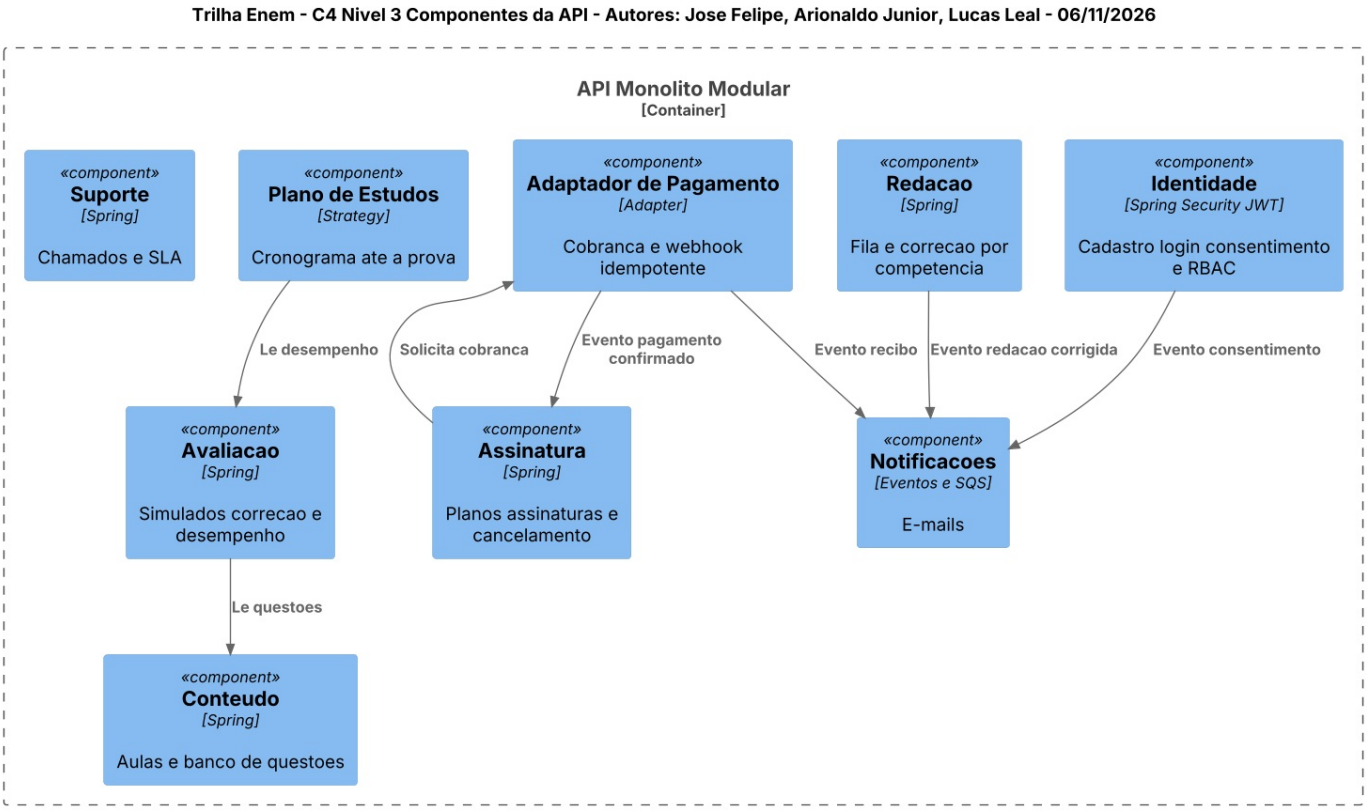
Como explicar: o sistema é uma caixa única com quatro tipos de pessoas e dois sistemas externos; o gateway fala nos dois sentidos (cobrança e webhook).

Nível 2: Contêineres



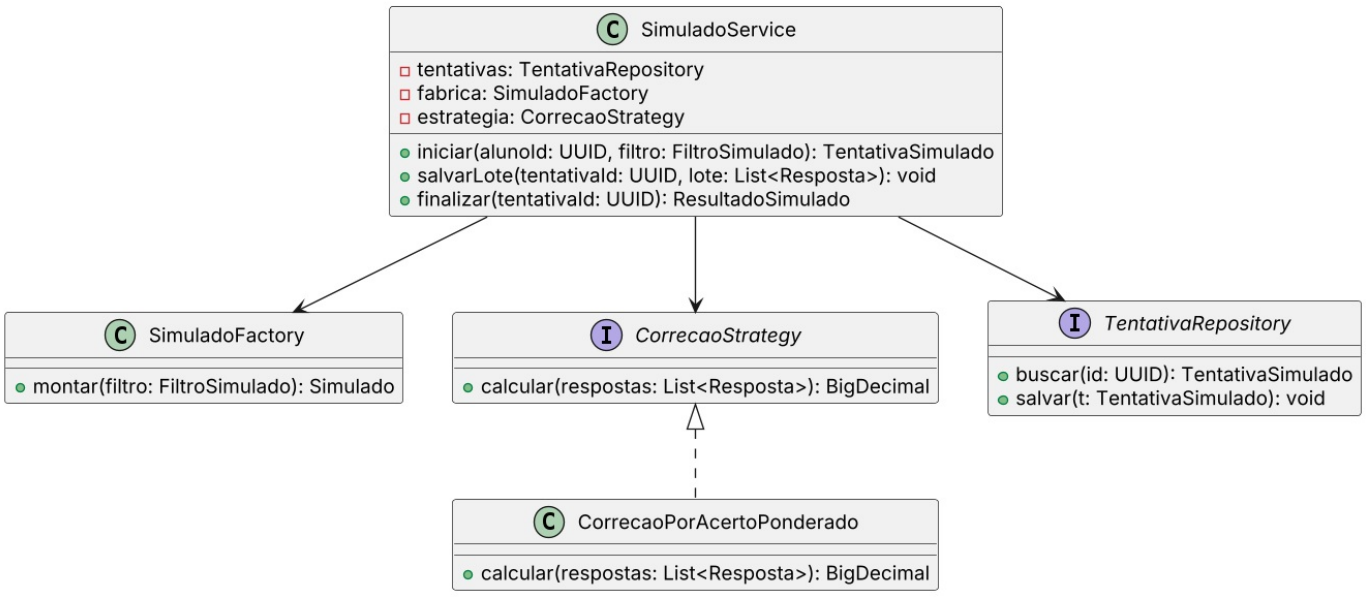
Como explicar: um único contêiner de regras (ADR-001), com banco, cache e fila gerenciados. O vídeo vai direto da CDN para o navegador e não passa pela API, para não gastar CPU das tarefas.

Nível 3: Componentes da API



Como explicar: nove módulos dentro do monolito. As setas mostram dependências em uma direção só e os eventos que desacoplam Pagamento, Redação e Identidade das Notificações.

Nível 4: Código de um componente crítico (Avaliação)



Como explicar: escolhemos a Avaliação porque concentra o pico de carga (RNF02). O Strategy permite trocar o cálculo da nota sem tocar no serviço, e o salvamento em lote (`salvarLote`) implementa o limite de 30 s do RNF12.

Bloco 4. Diagramas UML

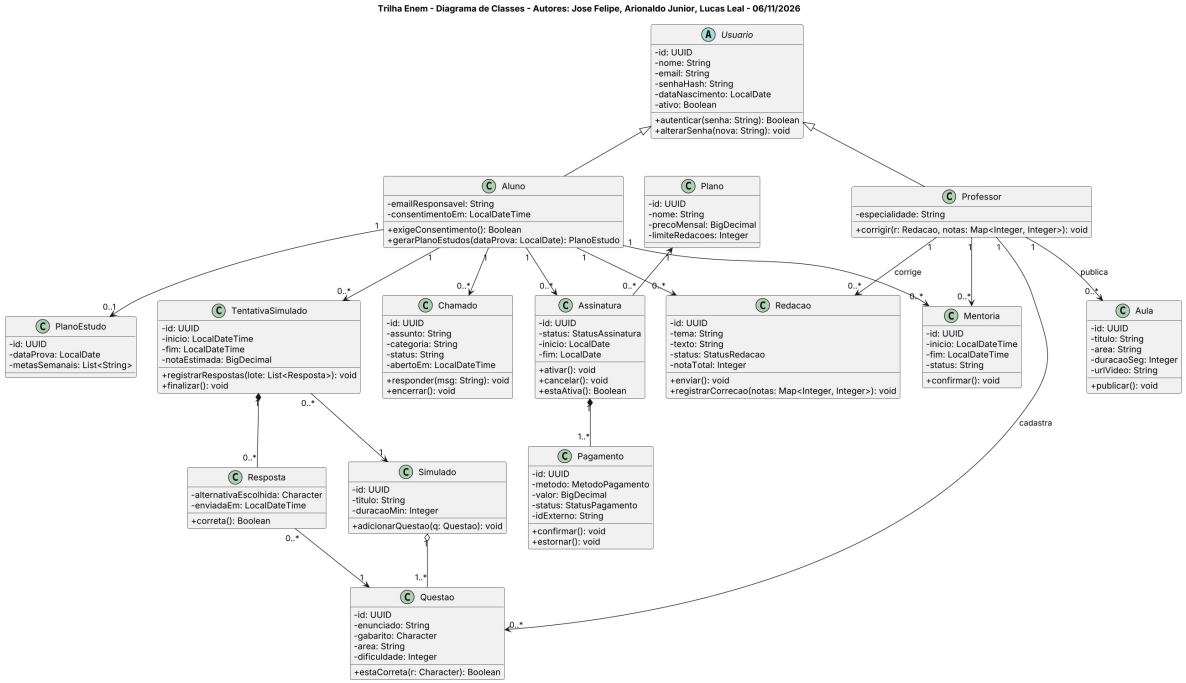
Seis tipos de diagrama (casos de uso, classes, sequência ×3, componentes, deployment, atividades ×2), todos com título, autores e data. Todos são apresentados e explicados ao vivo; por isso cada um traz um roteiro curto de fala.

4.1 Diagrama de Casos de Uso



Como explicar: (1) generalização de atores: todos herdam o Login. (2) *include* é obrigatório: assinar sempre efetua pagamento, e realizar simulado sempre gera simulado. (3) *extend* é opcional: o consentimento do responsável só ocorre quando o aluno é menor, e o estorno só quando há divergência. (4) PIX e cartão especializam Efetuar pagamento (generalização de casos de uso). Cobre RF001 a RF030.

4.2 Diagrama de Classes (15 classes, atributos tipados, visibilidade e relações)

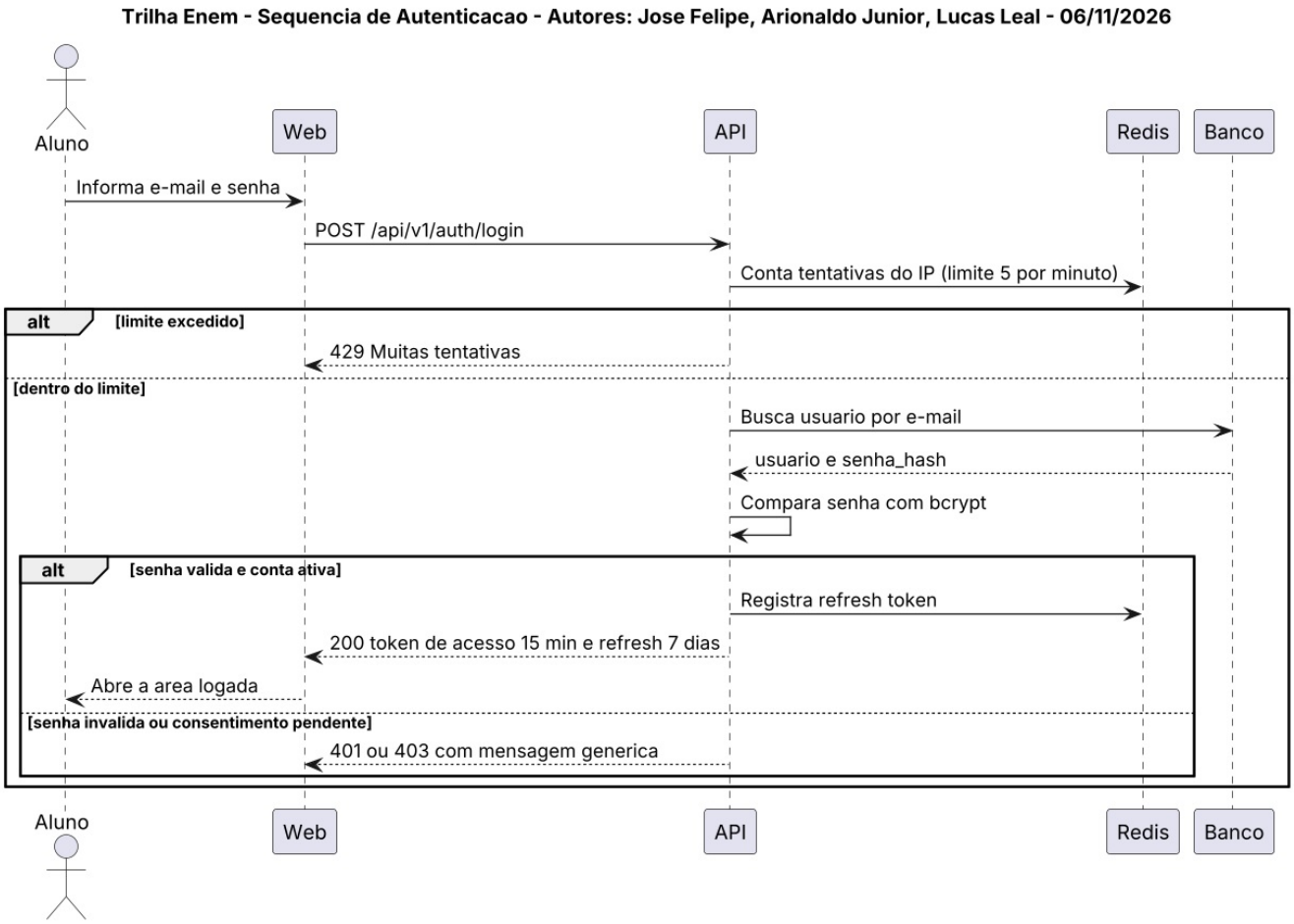


Decisões de modelagem (para a arguição):

- **Composição** Assinatura → Pagamento e Tentativa → Resposta: o filho não existe sem o pai. Um pagamento sempre pertence a uma assinatura e uma resposta sempre pertence a uma tentativa.
- **Agregação** Simulado → Questão: a questão existe independentemente e é reutilizada em vários simulados. Por isso não é composição.
- **Herança** só em Usuário → Aluno/Professor: há atributos e comportamento comuns (autenticar) e uma relação "é um". Administrador e Financeiro/Suporte não são subclasses: são papéis de permissão (RBAC, RF005). Assim evitamos uma hierarquia que cresce a cada novo perfil.

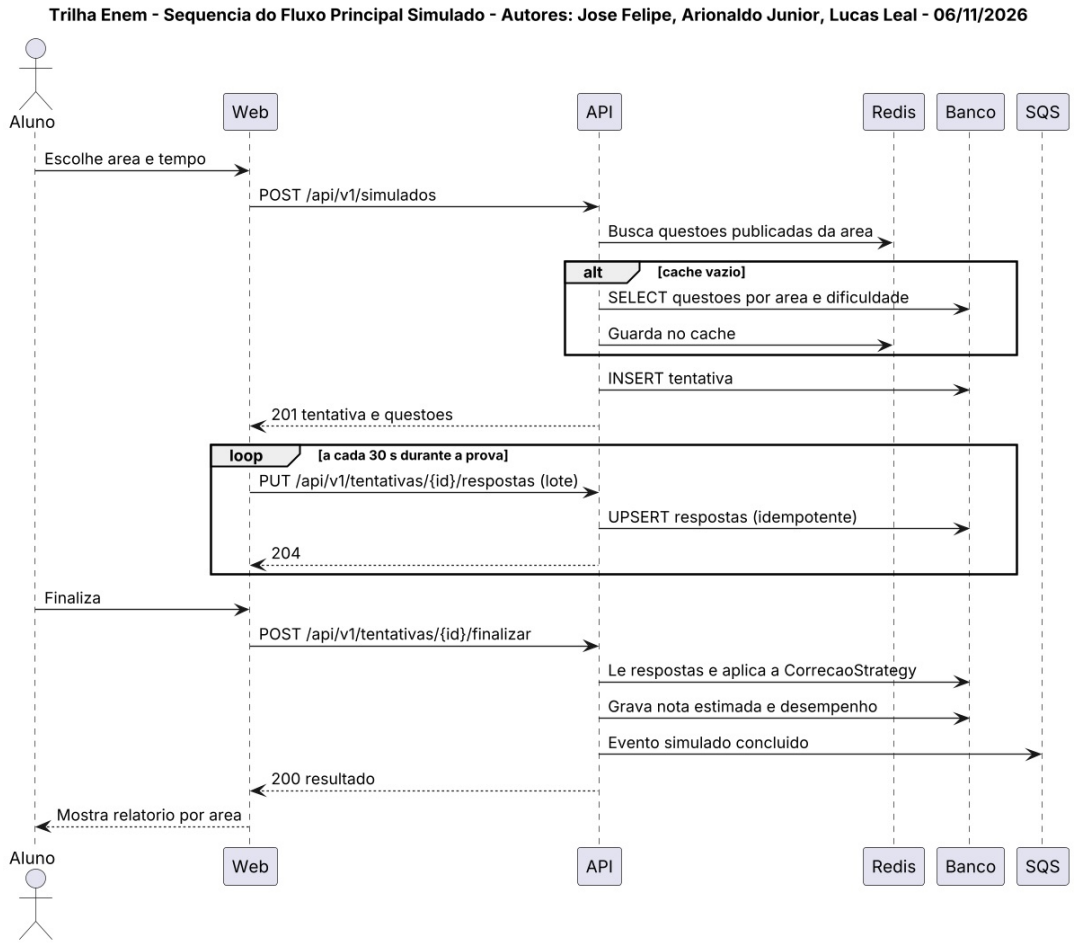
4.3 Diagramas de Sequência (3)

Sequência 1: Autenticação (RF003, RNF05)



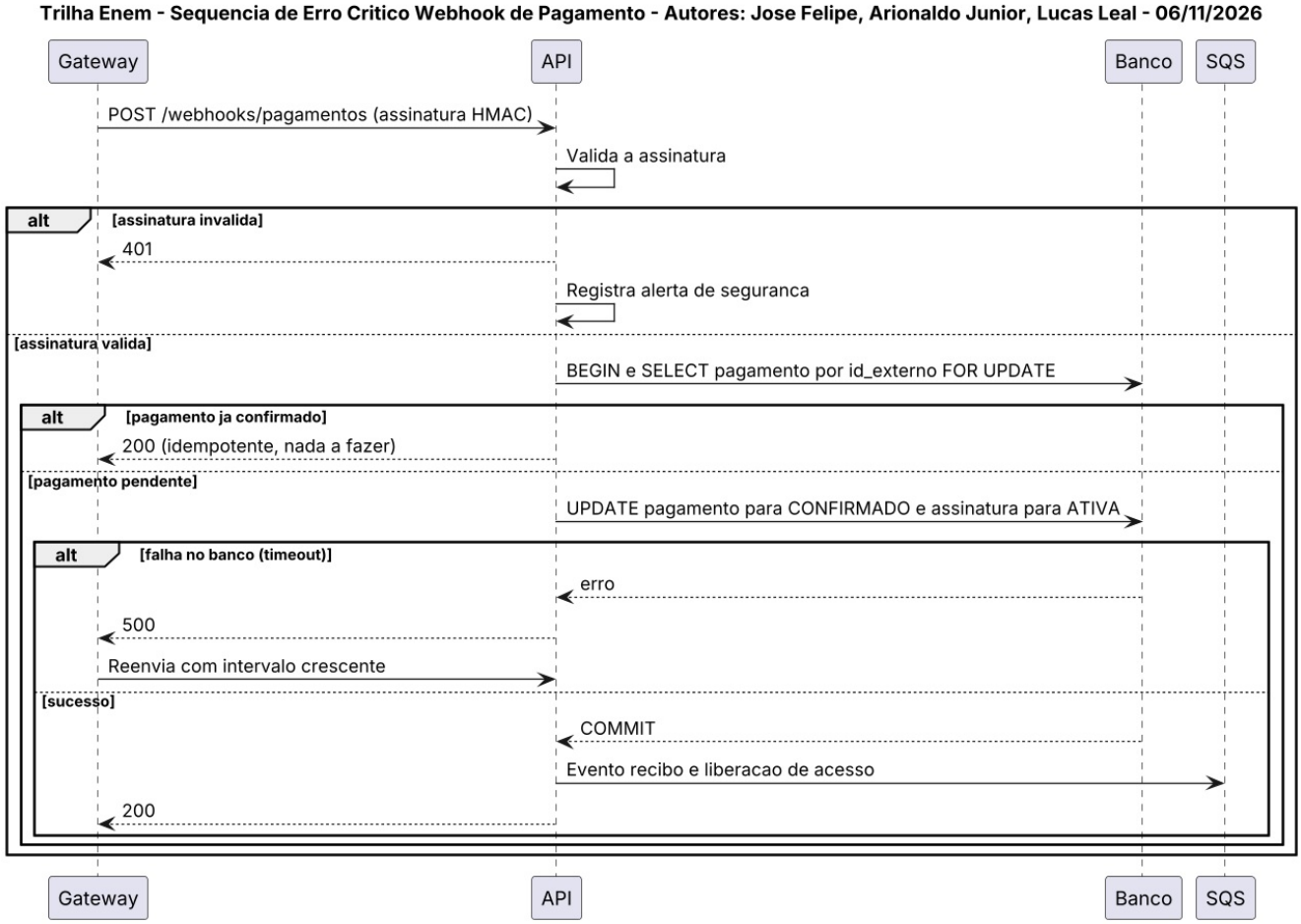
Como explicar: o limite por IP vem antes de consultar o banco, o que protege o banco de força bruta (OWASP A07). A mensagem de erro é genérica para não revelar se o e-mail existe.

Sequência 2: Fluxo principal (happy path), realizar simulado (RF017 a RF020)



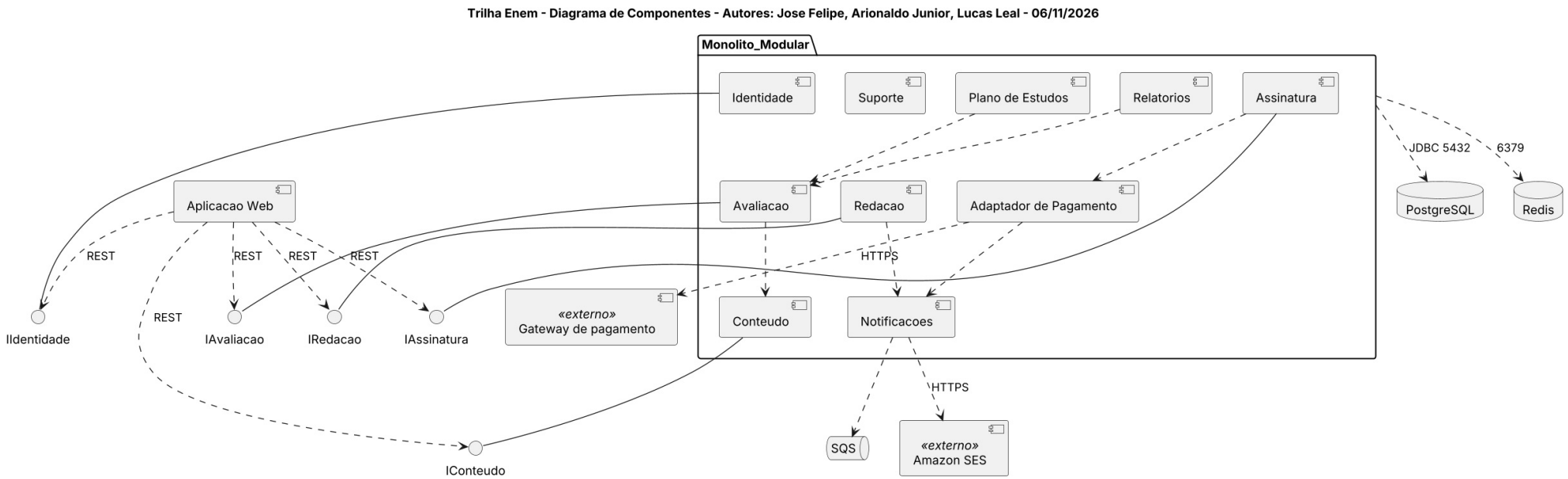
Como explicar: o cache evita que 6.000 alunos consultem o mesmo conjunto de questões no banco; o lote de 30 s reduz a escrita de 1.000 para ≈ 333 req/s; o UPSERT torna o reenvio seguro (RNF12).

Sequência 3: Cenário de erro crítico, webhook de pagamento duplicado ou com falha (RF012)



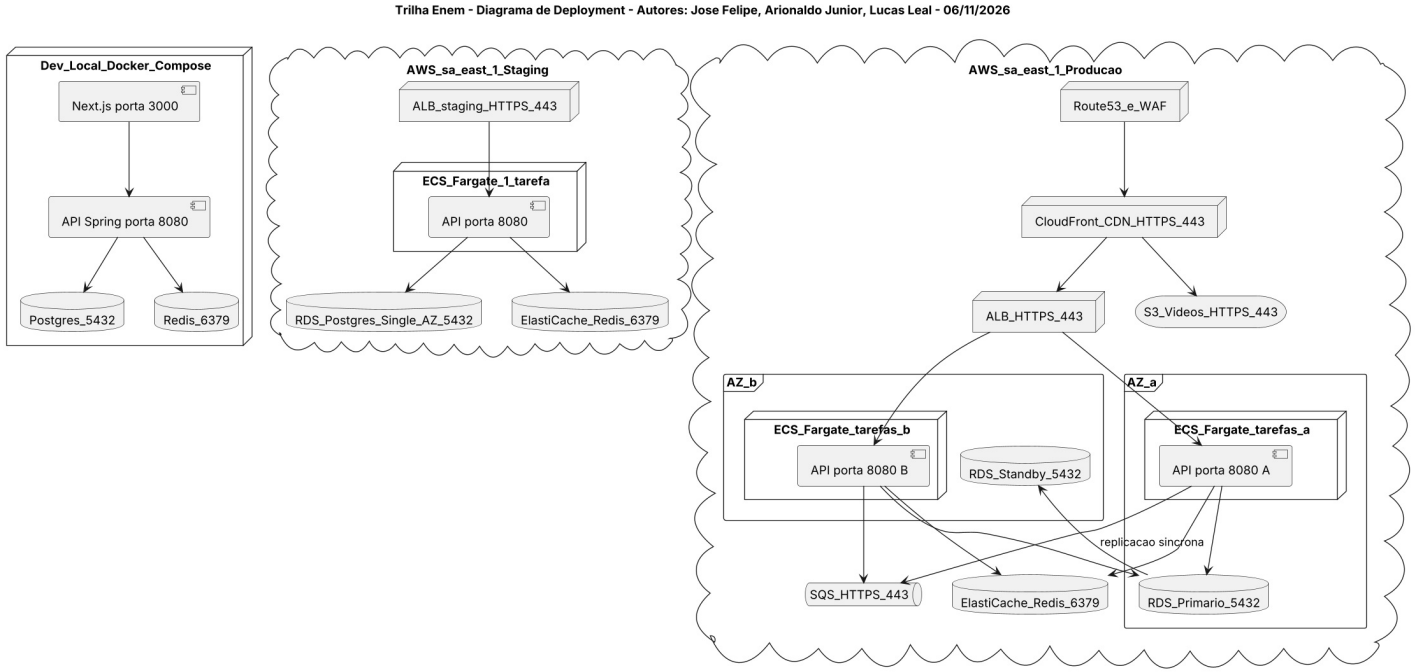
Como explicar: é o cenário mais crítico porque envolve dinheiro. Três proteções: assinatura HMAC (OWASP A08), trava de linha com `FOR UPDATE` e verificação do status antes de alterar (idempotência), e resposta 500 para o gateway tentar de novo quando o banco falha.

4.4 Diagrama de Componentes



Como explicar: cada módulo expõe uma interface e as dependências internas apontam em uma direção só (Avaliação depende de Conteúdo, nunca o contrário), o que mantém a possibilidade de extrair módulos no futuro (ADR-001).

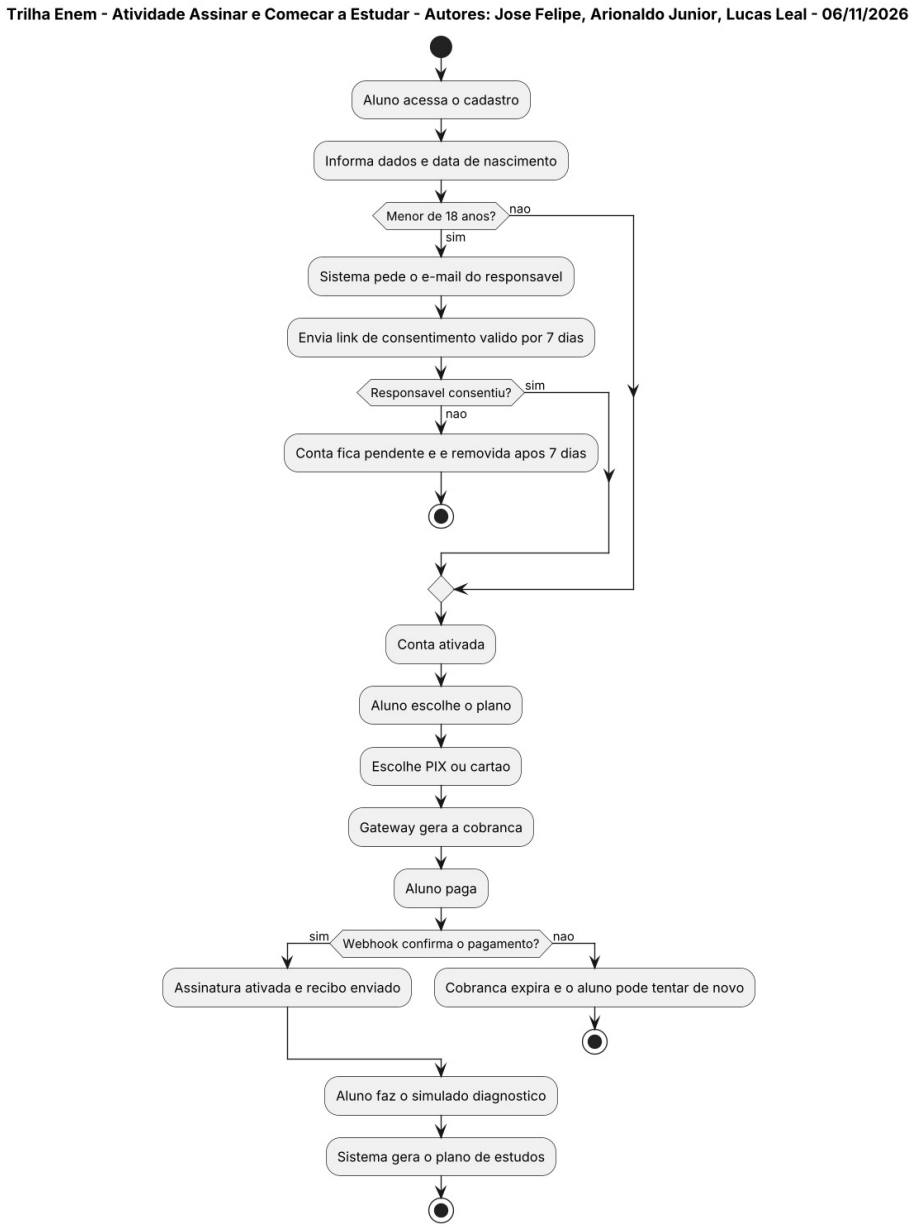
4.5 Diagrama de Deployment



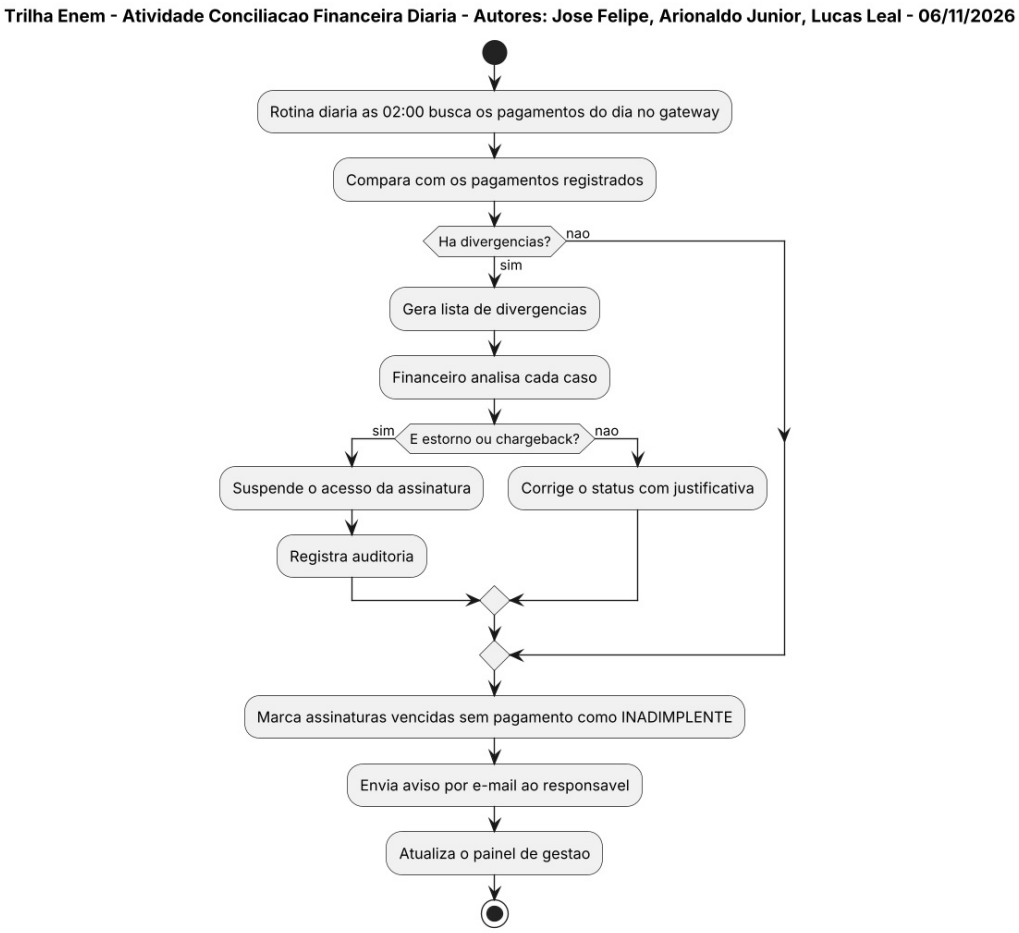
Como explicar: três ambientes. Dev roda tudo localmente em Docker Compose (custo zero). Staging é uma cópia reduzida da produção (1 tarefa, banco em zona única) para testar o deploy e o smoke test. Produção usa duas zonas de disponibilidade com tarefas em ambas e banco com standby síncrono; é isso que sustenta o RNF03 (99,9%). Portas: 443 (HTTPS), 8080 (API), 3000 (web em dev), 5432 (PostgreSQL), 6379 (Redis).

4.6 Diagramas de Atividades (2)

Atividade 1: processo de negócio principal, assinar e começar a estudar



Atividade 2: processo administrativo, conciliação financeira diária



Como explicar (as duas): a primeira mostra o caminho do dinheiro e do consentimento de ponta a ponta e conecta RF001, RF002, RF008, RF017 e RF024. A segunda garante que acesso liberado sempre bate com dinheiro recebido (persona Carla, RF012).

4.7 Rastreabilidade UML × requisitos

Diagrama	Cobre
Casos de uso	RF001 a RF030 (por ator)
Classes	Entidades do Bloco 5 (cada classe vira tabela)
Sequência 1, 2 e 3	RF003, RF017 a RF020, RF012
Componentes e C4	ADR-001 a ADR-011
Deployment	RNF02, RNF03, ADR-006
Atividades	RF001, RF002, RF008, RF012, RF024

Bloco 5. Banco de Dados

Responsável principal: Lucas Leal (DBA). A escolha do banco vem da arquitetura (ADR-001 monolito, ADR-004) e dos requisitos (RF012, RF020, RNF04, RNF06).

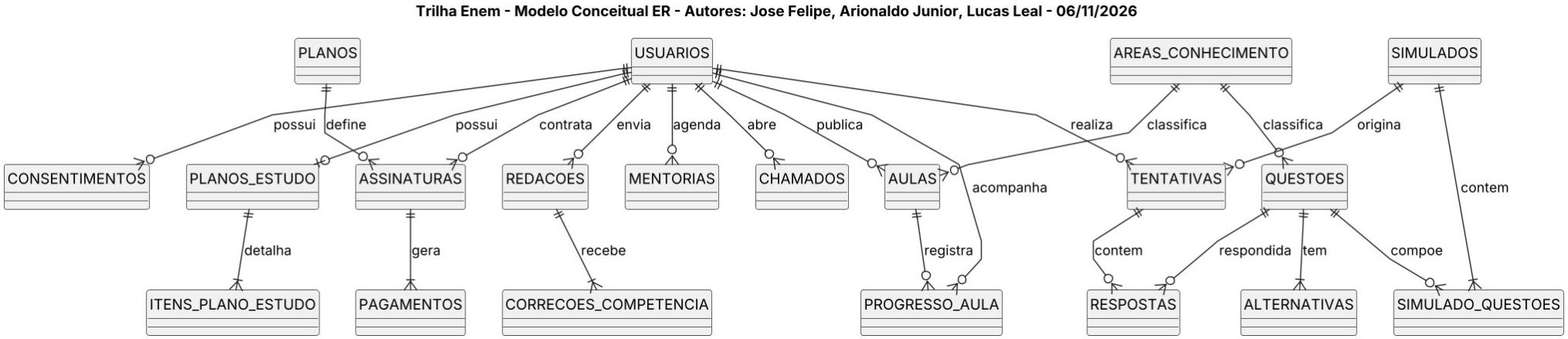
5.1 Decisão do banco

Critério (requisito de origem)	PostgreSQL 16 (escolhido)	MongoDB (descartado)	MySQL (descartado)
Integridade entre assinatura, pagamento e acesso (RF012)	Chaves estrangeiras, CHECK e UNIQUE parciais no próprio banco	Sem FKs; integridade fica no código	FKs e CHECK disponíveis
Transações em várias tabelas (webhook: pagamento + assinatura)	ACID completo	Transações multi-documento existem, com custo e limites	ACID completo
Relatórios de desempenho e painel (RF020, RF030)	Window functions, CTEs, views materializadas	Agregações possíveis, junções limitadas	Window functions e CTEs disponíveis, com menos recursos para views
Regras de negócio no esquema (sem dupla reserva de mentoria, uma assinatura ativa por aluno)	Exclusion constraint e índice único parcial	Precisa de código e lock	Sem exclusion constraint; índice parcial inexistente
LGPD (RNF06): criptografia, colunas sensíveis, auditoria	Criptografia em repouso (KMS), cifragem de coluna e row-level security	Criptografia em repouso, sem controle por linha nativo	Criptografia em repouso, sem row-level security nativo
Operação gerenciada em Multi-AZ (RNF03, RNF04)	RDS Multi-AZ com PITR	Atlas ou DocumentDB: serviço adicional	RDS Multi-AZ com PITR

Decisão: PostgreSQL em RDS Multi-AZ. O MongoDB perderia justamente o que o domínio mais precisa (integridade relacional e consultas agregadas). O MySQL funcionaria, mas não oferece as restrições declarativas que eliminam dois bugs críticos de negócio (reserva dupla e assinatura ativa duplicada).

5.2 Modelo Conceitual (ER)

Professores e alunos ficam na mesma entidade USUARIOS, diferenciados por papel (consistente com a herança do diagrama de classes, 4.2).



Cardinalidades-chave: uma assinatura tem 1 ou mais pagamentos (1:N, obrigatório); um simulado tem 1 ou mais questões e uma questão pode estar em vários simulados (N:N resolvido por SIMULADO_QUESTOES); cada redação recebe de 1 a 5 correções por competência (as cinco da rubrica); um aluno tem no máximo um plano de estudos ativo (1:0..1).

5.3 Modelo Lógico

Normalizado até a 3FN: cada atributo depende da chave, de toda a chave e somente dela. **Por que 3FN e não menos:** evita anomalias de atualização em dados financeiros (preço do plano e pagamentos) e em questões compartilhadas por vários simulados. **Por que não além (desnormalizar):** o desempenho de leitura dos relatórios é resolvido com a view materializada do item 5.4, e não duplicando colunas. **Exceção consciente:** `redacoes.nota_total` é valor derivado, gravado na mesma transação das cinco notas, para ordenar e filtrar sem somar a cada consulta.

Tabela	Chave primária	Chaves estrangeiras	Índices e justificativa
usuarios	id (uuid)	—	UNIQUE(email): busca de login, alvo do RNF01
consentimentos	id	aluno_id → usuarios	UNIQUE(token_hash): validar link de consentimento sem expor o token
planos	id	—	UNIQUE(nome)
assinaturas	id	aluno_id → usuarios; plano_id → planos	UNIQUE parcial (aluno_id) onde status é PENDENTE ou ATIVA: impede duas assinaturas ativas; índice em aluno_id para o login
pagamentos	id	assinatura_id → assinaturas	UNIQUE(id_externo): garante idempotência do webhook (RF012); índice em assinatura_id
areas_conhecimento	id (smallint)	—	Tabela de domínio com as 4 áreas do Enem e a redação
aulas	id	professor_id → usuarios; area_id → areas	Índice parcial (area_id) onde publicada: listagem por área (RF014)
progresso_aula	(aluno_id, aula_id)	aluno_id; aula_id	PK composta: um registro por aluno e aula, permite UPSERT
questoes	id	area_id; professor_id	Índice parcial (area_id, dificuldade) onde publicada: geração do simulado (RF017)
alternativas	(questao_id, letra)	questao_id	PK composta evita alternativa duplicada
simulados	id	criado_por → usuarios	—
simulado_questoes	(simulado_id, questao_id)	simulado_id; questao_id	UNIQUE(simulado_id, ordem): ordem sem repetição
tentativas	id	aluno_id; simulado_id	Índice (aluno_id, inicio DESC): histórico do aluno
respostas	(tentativa_id, questao_id)	tentativa_id (cascade); questao_id	PK composta permite INSERT ... ON CONFLICT DO UPDATE: o reenvio de lote é idempotente (RNF12)
redacoes	id	aluno_id; professor_id	Índice parcial (status, prazo_em) onde status não é CORRIGIDA: fila de correção ordenada por prazo (RF023)
correcoes_competencia	(redacao_id, competencia)	redacao_id (cascade)	PK composta: uma nota por competência
mentorias	id	aluno_id; professor_id	EXCLUDE: um professor nunca tem dois horários sobrepostos
chamados	id	aluno_id; atendente_id	Índice (status, aberto_em): fila do suporte (RF029)
planos_estudo	id	aluno_id (UNIQUE)	UNIQUE(aluno_id): máximo um plano por aluno
itens_plano_estudo	(plano_id, semana, area_id)	plano_id (cascade); area_id	PK composta
auditoria	id (bigint identity)	—	Índice (entidade, entidade_id): trilha de auditoria da LGPD

5.4 Modelo Físico (DDL executável, PostgreSQL 16)

O script abaixo foi **executado em um PostgreSQL 16 limpo** (21 tabelas e a view materializada criadas sem erro) e as regras de negócio foram testadas com inserts: segunda assinatura ativa do mesmo aluno é rejeitada; mentorias sobrepostas do mesmo professor são rejeitadas (e horários adjacentes passam); `id_externo` duplicado em pagamentos é rejeitado; redação CORRIGIDA sem nota é rejeitada; reenvio de resposta atualiza em vez de duplicar. O arquivo completo acompanha a entrega (`schema.sql`). Optou-se por `varchar` com CHECK em vez de ENUM, porque alterar valores de ENUM em produção é mais trabalhoso que trocar um CHECK em uma migration.

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE areas_conhecimento (
  id smallint PRIMARY KEY,
  nome varchar(40) NOT NULL UNIQUE
);
INSERT INTO areas_conhecimento (id, nome) VALUES
  (1, 'Linguagens'), (2, 'Ciencias Humanas'), (3, 'Ciencias da Natureza'),
  (4, 'Matematica'), (5, 'Redacao');

CREATE TABLE usuarios (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  nome varchar(120) NOT NULL,
  email varchar(254) NOT NULL UNIQUE,
  senha_hash varchar(100) NOT NULL,
  data_nascimento date NOT NULL CHECK (data_nascimento > DATE '1900-01-01'),
  papel varchar(20) NOT NULL
    CHECK (papel IN ('ALUNO','PROFESSOR','ADMINISTRADOR','SUPORTE_FINANCEIRO')),
  ativo boolean NOT NULL DEFAULT false,
  criado_em timestamptz NOT NULL DEFAULT now(),
  anonimizado_em timestamptz
);

CREATE TABLE consentimentos (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  aluno_id uuid NOT NULL REFERENCES usuarios(id),
  email_responsavel varchar(254) NOT NULL,
  versao_termo varchar(20) NOT NULL,
  token_hash char(64) NOT NULL UNIQUE,
  enviado_em timestamptz NOT NULL DEFAULT now(),
  expira_em timestamptz NOT NULL,
  aceito_em timestamptz,
  ip_aceite inet,
  CHECK (expira_em > enviado_em)
);

CREATE TABLE planos (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  nome varchar(60) NOT NULL UNIQUE,
  preco_mensal numeric(10,2) NOT NULL CHECK (preco_mensal > 0),
```

```
  limite_redacoes smallint NOT NULL DEFAULT 0 CHECK (limite_redacoes >= 0),
  ativo boolean NOT NULL DEFAULT true
);

CREATE TABLE assinaturas (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  aluno_id uuid NOT NULL REFERENCES usuarios(id),
  plano_id uuid NOT NULL REFERENCES planos(id),
  status varchar(12) NOT NULL DEFAULT 'PENDENTE'
    CHECK (status IN ('PENDENTE','ATIVA','CANCELADA','INADIMPLENTE')),
  inicio date,
  fim date,
  criada_em timestamptz NOT NULL DEFAULT now(),
  CHECK (fim IS NULL OR inicio IS NULL OR fim >= inicio)
);
CREATE UNIQUE INDEX ux_assinatura_ativa ON assinaturas (aluno_id)
  WHERE status IN ('PENDENTE','ATIVA');
CREATE INDEX ix_assinaturas_aluno ON assinaturas (aluno_id);

CREATE TABLE pagamentos (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  assinatura_id uuid NOT NULL REFERENCES assinaturas(id),
  metodo varchar(10) NOT NULL CHECK (metodo IN ('PIX','CARTAO')),
  valor numeric(10,2) NOT NULL CHECK (valor > 0),
  status varchar(12) NOT NULL DEFAULT 'PENDENTE'
    CHECK (status IN ('PENDENTE','CONFIRMADO','ESTORNADO','EXPIRADO')),
  id_externo varchar(64) NOT NULL UNIQUE,
  cpf_pagador_cifrado bytea,
  criado_em timestamptz NOT NULL DEFAULT now(),
  confirmado_em timestamptz
);
CREATE INDEX ix_pagamentos_assinatura ON pagamentos (assinatura_id);

CREATE TABLE aulas (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  professor_id uuid NOT NULL REFERENCES usuarios(id),
  area_id smallint NOT NULL REFERENCES areas_conhecimento(id),
  titulo varchar(150) NOT NULL,
  duracao_seg integer NOT NULL CHECK (duracao_seg > 0),
  url_video varchar(500) NOT NULL,
  publicada boolean NOT NULL DEFAULT false,
  criada_em timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX ix_aulas_area_publicada ON aulas (area_id) WHERE publicada;

CREATE TABLE progresso_aula (
  aluno_id uuid NOT NULL REFERENCES usuarios(id),
  aula_id uuid NOT NULL REFERENCES aulas(id),
  posicao_seg integer NOT NULL DEFAULT 0 CHECK (posicao_seg >= 0),
  concluida boolean NOT NULL DEFAULT false,
  atualizado_em timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (aluno_id, aula_id)
);

CREATE TABLE questoes (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  area_id smallint NOT NULL REFERENCES areas_conhecimento(id),
  professor_id uuid NOT NULL REFERENCES usuarios(id),
  enunciado text NOT NULL,
  gabarito char(1) NOT NULL CHECK (gabarito IN ('A','B','C','D','E')),
  dificuldade smallint NOT NULL CHECK (dificuldade BETWEEN 1 AND 5),
  publicada boolean NOT NULL DEFAULT false,
  criada_em timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX ix_questoes_area_dif ON questoes (area_id, dificuldade) WHERE publicada;

CREATE TABLE alternativas (
  questao_id uuid NOT NULL REFERENCES questoes(id) ON DELETE CASCADE,
  letra char(1) NOT NULL CHECK (letra IN ('A','B','C','D','E')),
  texto text NOT NULL,
  PRIMARY KEY (questao_id, letra)
);

CREATE TABLE simulados (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  titulo varchar(120) NOT NULL,
  duracao_min smallint NOT NULL CHECK (duracao_min > 0),
  criado_por uuid REFERENCES usuarios(id),
  criado_em timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE simulado_questoes (
  simulado_id uuid NOT NULL REFERENCES simulados(id) ON DELETE CASCADE,
  questao_id uuid NOT NULL REFERENCES questoes(id),
  ordem smallint NOT NULL CHECK (ordem > 0),
  PRIMARY KEY (simulado_id, questao_id),
  UNIQUE (simulado_id, ordem)
);

CREATE TABLE tentativas (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  aluno_id uuid NOT NULL REFERENCES usuarios(id),
  simulado_id uuid NOT NULL REFERENCES simulados(id),
  inicio timestamptz NOT NULL DEFAULT now(),
  fim timestamptz,
  nota_estimada numeric(5,1),
  CHECK (fim IS NULL OR fim >= inicio)
);
CREATE INDEX ix_tentativas_aluno ON tentativas (aluno_id, inicio DESC);

CREATE TABLE respostas (
  tentativa_id uuid NOT NULL REFERENCES tentativas(id) ON DELETE CASCADE,
  questao_id uuid NOT NULL REFERENCES questoes(id),
  alternativa char(1) CHECK (alternativa IN ('A','B','C','D','E')),
  enviada_em timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (tentativa_id, questao_id)
);

CREATE TABLE redacoes (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  aluno_id uuid NOT NULL REFERENCES usuarios(id),
  professor_id uuid REFERENCES usuarios(id),
  tema varchar(200) NOT NULL,
  texto text NOT NULL CHECK (char_length(texto) BETWEEN 100 AND 8000),
  status varchar(12) NOT NULL DEFAULT 'ENVIADA'
    CHECK (status IN ('ENVIADA','EM_CORRECAO','CORRIGIDA')),
  enviada_em timestamptz NOT NULL DEFAULT now(),
  prazo_em timestamptz NOT NULL,
```

```
  corrigida_em timestamptz,
  nota_total smallint CHECK (nota_total BETWEEN 0 AND 1000),
  CHECK (status <> 'CORRIGIDA' OR (nota_total IS NOT NULL AND corrigida_em IS NOT NULL))
);
CREATE INDEX ix_redacoes_fila ON redacoes (status, prazo_em) WHERE status <> 'CORRIGIDA';
CREATE INDEX ix_redacoes_aluno ON redacoes (aluno_id);

CREATE TABLE correcoes_competencia (
  redacao_id uuid NOT NULL REFERENCES redacoes(id) ON DELETE CASCADE,
  competencia smallint NOT NULL CHECK (competencia BETWEEN 1 AND 5),
  nota smallint NOT NULL CHECK (nota BETWEEN 0 AND 200),
  comentario text,
  PRIMARY KEY (redacao_id, competencia)
);

CREATE TABLE mentorias (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  aluno_id uuid NOT NULL REFERENCES usuarios(id),
  professor_id uuid NOT NULL REFERENCES usuarios(id),
  inicio timestamptz NOT NULL,
  fim timestamptz NOT NULL,
  status varchar(12) NOT NULL DEFAULT 'AGENDADA'
    CHECK (status IN ('AGENDADA','CONFIRMADA','CANCELADA','REALIZADA')),
  CHECK (fim > inicio),
  EXCLUDE USING gist (professor_id WITH =, tstzrange(inicio, fim) WITH &&)
    WHERE (status <> 'CANCELADA')
);

CREATE TABLE chamados (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  aluno_id uuid NOT NULL REFERENCES usuarios(id),
  atendente_id uuid REFERENCES usuarios(id),
  assunto varchar(150) NOT NULL,
  categoria varchar(20) NOT NULL
    CHECK (categoria IN ('PAGAMENTO','CONTEUDO','CONTA','OUTRO')),
  status varchar(14) NOT NULL DEFAULT 'ABERTO'
    CHECK (status IN ('ABERTO','EM_ATENDIMENTO','ENCERRADO')),
  aberto_em timestamptz NOT NULL DEFAULT now(),
  primeira_resposta_em timestamptz,
  encerrado_em timestamptz
);
CREATE INDEX ix_chamados_fila ON chamados (status, aberto_em);

CREATE TABLE planos_estudo (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  aluno_id uuid NOT NULL UNIQUE REFERENCES usuarios(id),
  data_prova date NOT NULL,
  gerado_em timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE itens_plano_estudo (
  plano_id uuid NOT NULL REFERENCES planos_estudo(id) ON DELETE CASCADE,
  semana smallint NOT NULL CHECK (semana > 0),
  area_id smallint NOT NULL REFERENCES areas_conhecimento(id),
  meta varchar(200) NOT NULL,
  concluida boolean NOT NULL DEFAULT false,
  PRIMARY KEY (plano_id, semana, area_id)
);

CREATE TABLE auditoria (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  usuario_id uuid,
  acao varchar(40) NOT NULL,
  entidade varchar(40),
  entidade_id uuid,
  ip inet,
  ocorrido_em timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX ix_auditoria_entidade ON auditoria (entidade, entidade_id);

CREATE MATERIALIZED VIEW mv_desempenho_aluno_area AS
SELECT t.aluno_id,
       q.area_id,
       count(*) AS respondidas,
       count(*) FILTER (WHERE r.alternativa = q.gabarito) AS acertos
FROM respostas r
JOIN tentativas t ON t.id = r.tentativa_id
JOIN questoes q ON q.id = r.questao_id
WHERE t.fim IS NOT NULL
GROUP BY t.aluno_id, q.area_id;
CREATE UNIQUE INDEX ux_mv_desempenho ON mv_desempenho_aluno_area (aluno_id, area_id);
```

Pontos para explicar: (1) a view materializada alimenta o relatório do aluno (RF020) sem recalcular acertos a cada acesso; atualiza-se com `REFRESH MATERIALIZED VIEW CONCURRENTLY` a cada 5 minutos, sem bloquear leituras. É o nosso "CQRS leve" (ADR 3.3). (2) `ON DELETE CASCADE` só onde o filho não faz sentido sem o pai e não há obrigação legal de guardar (respostas, itens do plano, notas por competência). Pagamentos não têm cascade: a composição do UML vale conceitualmente, mas o registro financeiro precisa ser mantido por obrigação fiscal, então a exclusão é lógica (anonimização). (3) o EXCLUDE impede dois horários sobrepostos para o mesmo professor, mesmo com duas requisições simultâneas.

5.5 Dicionário de Dados (entidades com dados pessoais e entidades críticas)

PII = dado pessoal (LGPD). "Tratamento" indica como o campo é protegido.

usuarios

Campo	Tipo	Restrições	Descrição de negócio	PII / tratamento
id	uuid	PK	Identificador	Não
nome	varchar(120)	NOT NULL	Nome completo	Sim: anonimizado na exclusão
email	varchar(254)	NOT NULL, UNIQUE	Login e contato	Sim: anonimizado (hash) na exclusão
senha_hash	varchar(100)	NOT NULL	Senha com bcrypt custo ≥ 12	Sim: nunca exportada
data_nascimento	date	NOT NULL	Define se há consentimento do responsável	Sim: mantida só para regra de idade
papel	varchar(20)	NOT NULL, CHECK	Perfil de acesso (RBAC)	Não
ativo	boolean	NOT NULL	Conta liberada (consentimento dado quando menor)	Não
anonimizado_em	timestamptz	—	Data da anonimização (RF006)	Não

consentimentos

Campo	Tipo	Restrições	Descrição	PII / tratamento
email_responsavel	varchar(254)	NOT NULL	Contato do responsável	Sim: anonimizado após a exclusão da conta
versao_termo	varchar(20)	NOT NULL	Versão do termo aceito (prova de consentimento)	Não
token_hash	char(64)	NOT NULL, UNIQUE	Hash do token do link	Não: só o hash é guardado
aceito_em	timestamptz	—	Momento do aceite	Não
ip_aceite	inet	—	IP do aceite (prova)	Sim: retido pelo prazo do termo

pagamentos

Campo	Tipo	Restrições	Descrição	PII / tratamento
id_externo	varchar(64)	NOT NULL, UNIQUE	Identificador no gateway (idempotência)	Não
valor	numeric(10,2)	> 0	Valor cobrado	Não
status	varchar(12)	CHECK	PENDENTE, CONFIRMADO, ESTORNADO, EXPIRADO	Não
cpf_pagador_cifrado	bytea	—	CPF do pagador (exigido pelo PIX), cifrado com AES-256-GCM e chave no KMS	Sim: cifrado em coluna; mantido pelo prazo fiscal

redacoes e correcoes_competencia

Campo	Tipo	Restrições	Descrição	PII / tratamento
texto	text	100 a 8.000 caracteres	Texto do aluno	Sim: pode conter dados pessoais; excluído com a conta
status	varchar(12)	CHECK	ENVIADA, EM_CORRECAO, CORRIGIDA	Não
prazo_em	timestamptz	NOT NULL	Prazo de 72 h (RNF11)	Não
nota_total	smallint	0 a 1.000	Soma das 5 competências	Não
competencia / nota	smallint	1 a 5 / 0 a 200	Nota por competência do Enem	Não

Demais entidades (campos-chave). Todas as colunas de planos, assinaturas, aulas, questoes, simulados, tentativas, respostas, mentorias, chamados e auditoria estão tipadas e restringidas no DDL do item 5.4. Para o dicionário completo, adicionar `COMMENT ON COLUMN` a cada coluna e exportar com o pgAdmin ou o SchemaSpy. Campos PII adicionais: `chamados.assunto` (texto livre, pode conter dados pessoais) e `auditoria.ip` (retido por 6 meses).

5.6 Estratégia de dados

Backup (RPO ≤ 5 min, RTO ≤ 30 min: RNF04).

Item	Frequência	Retenção	Justificativa
Backup automático com recuperação ponto no tempo (PITR) do RDS	Contínuo (logs de transação) + snapshot diário	14 dias	Permite voltar a qualquer minuto dos últimos 14 dias; o RPO fica em minutos
Snapshot manual de segurança	Semanal	90 dias	Proteção contra erro humano descoberto tardiamente
Dados financeiros (pagamentos)	Exportação mensal cifrada para S3	5 anos (a confirmar com contador) [COTAR]	Obrigação fiscal; o prazo exato deve ser confirmado
Teste de restauração	Mensal, em staging	—	O RTO só vale se for medido

LGPD (RNF06).

- Campos PII marcados no dicionário (5.5). Minimização: o CPF só é coletado no pagamento e fica cifrado em coluna; para menores, o CPF é do responsável.
- Criptografia em repouso (RDS com KMS) e em trânsito (TLS 1.2+). Senhas só como hash.
- Exclusão (RF006): anonimização de nome, email e textos livres, mantendo pagamentos pelo prazo fiscal. Prazo de resposta ao titular: 15 dias.
- Retenção: contas inativas por 24 meses recebem aviso e, sem resposta em 30 dias, são anonimizadas (prazo de política a validar com o jurídico).
- Trilha de auditoria (`auditoria`) para acessos de professor e suporte a dados de aluno.

Migrations sem downtime.

- Ferramenta: Flyway, com scripts versionados (`V1__criar_usuarios.sql`) executados pelo pipeline antes do deploy da aplicação.
- Padrão *expand/contract*: (1) adicionar coluna nullable, (2) publicar a versão que grava nas duas colunas, (3) preencher o histórico em lotes, (4) trocar a leitura, (5) remover a coluna antiga em outra release. A versão N-1 da aplicação deve funcionar com o esquema N durante o deploy gradual do ECS.
- Índices novos com `CREATE INDEX CONCURRENTLY` para não bloquear escritas.
- Toda migration roda antes em staging com uma cópia anonimizada de produção.

Bloco 6. Infraestrutura e DevOps

Responsável principal: Lucas Leal. O bloco é opcional no enunciado, mas a infra aparece na apresentação e na arguição (SLA de 99,9%, "e se o banco cair?"), então ele está completo. Regra: custo inventado zera o bloco, por isso a tabela de custos separa o que foi calculado com preço público consultado do que ainda precisa da calculadora oficial.

6.1 Diagrama de infraestrutura

O diagrama de deployment do item 4.5 mostra os ambientes. Complemento de rede (produção, região sa-east-1, duas zonas de disponibilidade):

Camada	Faixa / recurso	O que roda	Regra de acesso (security group)
VPC	10.0.0.0/16	Rede isolada	—
Sub-redes públicas (AZ a e b)	10.0.0.0/24 e 10.0.1.0/24	Application Load Balancer e NAT Gateway	ALB: 443 vindo da internet (via CloudFront)
Sub-redes privadas de aplicação	10.0.10.0/24 e 10.0.11.0/24	Tarefas ECS Fargate (API 8080, Web 3000)	Só aceita do ALB
Sub-redes privadas de dados	10.0.20.0/24 e 10.0.21.0/24	RDS PostgreSQL (5432) e ElastiCache Redis (6379)	Só aceita das tarefas de aplicação
Borda	Route 53, WAF, CloudFront	DNS, filtro de ataques e CDN	WAF com regras gerenciadas e limite de taxa
Serviços gerenciados	S3, SQS, SES, Secrets Manager, CloudWatch	Vídeo, fila, e-mail, segredos e logs	Acesso por papéis IAM das tarefas

Por que sub-redes privadas para aplicação e dados: o banco e as tarefas não têm IP público, o que reduz a superfície de ataque (OWASP A05). O custo é o NAT Gateway para as tarefas falarem com serviços externos (item 6.2).

6.2 Custo mensal de infraestrutura por ambiente

Método. Cada linha mostra a conta feita. Os preços unitários vêm de fontes públicas consultadas em 07/11/2026; as linhas marcadas **[COTAR]** não tiveram preço verificado e precisam ser lidas em calculator.aws (região South America, São Paulo), com o print anexado ao AID. Conversão para reais: usar a cotação do dia [COTAR]. Mês de referência: 730 horas.

Preços unitários usados [F]:

- Fargate Linux/x86 em São Paulo: US$ 0,0696 por vCPU-hora e US$ 0,0076 por GB-hora (CloudForecast, guia de preços do ECS; a data de coleta do autor não é informada, então confirmar na calculadora).
- RDS PostgreSQL db.t4g.medium em São Paulo: US$ 0,137/h em zona única e US$ 0,275/h em Multi-AZ (Spare Cores; o valor mensal em zona única, ≈ US$ 100, coincide com o da Bytebase).

Produção

Linha | Cálculo | US$/mês |
|—|—|—| | API: 2 tarefas fixas de 1 vCPU / 2 GB (duas AZ) | (1 × 0,0696 + 2 × 0,0076) = 0,0848/h × 730 × 2 | 123,81 | | API: reforço de pico, dimensionado até o teto de +6 tarefas por 40 h/mês [P] | 0,0848 × 6 × 40 | 20,35 | | Web Next.js: 2 tarefas de 0,5 vCPU / 1 GB | (0,5 × 0,0696 + 1 × 0,0076) = 0,0424/h × 730 × 2 | 61,90 | | RDS PostgreSQL db.t4g.medium Multi-AZ | 0,275 × 730 | 200,75 | | **Subtotal calculado (computação e instância do banco)** | | **406,81** | | Armazenamento do RDS: 100 GB gp3 + backup excedente | [COTAR] | a cotar | | ElastiCache Redis (nó pequeno, 1 nó) | [COTAR] | a cotar | | Application Load Balancer (730 h + LCU). Referência: US$ 0,0225/h na Virgínia, e São Paulo costuma ser mais caro | [COTAR] | a cotar | | NAT Gateway × 2 (um por AZ) | [COTAR] | a cotar | | S3: 450 GB de vídeo (P08) | [COTAR] | a cotar | | CloudFront: ≈ 14.000 GB/mês de saída (P07) | [COTAR] | a cotar (provável maior linha variável, risco R05) | | SQS, SES, Route 53, WAF, Secrets Manager, CloudWatch | [COTAR] | a cotar |

Staging (cópia reduzida, mesmo desenho, sem pico)

Linha	Cálculo	US$/mês
API: 1 tarefa de 0,5 vCPU / 1 GB	0,0424 × 730	30,95
Web: 1 tarefa de 0,5 vCPU / 1 GB	0,0424 × 730	30,95
RDS db.t4g.medium, zona única	0,137 × 730	100,01
Subtotal calculado		**161,91**
ALB, NAT (1), Redis, S3, logs	[COTAR]	a cotar

Dev: roda em Docker Compose na máquina de cada desenvolvedor. Custo de nuvem: US$ 0.

Total já calculado (produção + staging): US$ 568,72/mês, sem as linhas a cotar. O total final do AID só vale depois de preencher as linhas [COTAR] e converter para reais.

Por que esses tamanhos: 2 tarefas de 1 vCPU respeitam a alta disponibilidade (uma por AZ) e entregam ≈ 500 req/s fixos (P06); o autoscaling sobe até 8 tarefas, o que cobre ≈ 2.000 req/s, o dobro do pico de 1.000 req/s (margem para RNF02). O custo do reforço foi calculado pelo teto (+6 tarefas), que é conservador: o pico esperado de P05/P06 pede 4 tarefas no total. O banco db.t4g.medium (2 vCPU, 4 GB) é o ponto mais frágil: é uma instância "burstable", e o pico de simulado pode esgotar os créditos de CPU. **Gatilho de escala (documentado):** CPU do banco acima de 70% por 15 minutos, ou créditos zerados, leva a migrar para uma instância de uso geral maior e criar réplica de leitura para relatórios [COTAR]. Otimização possível: tarefas ARM (Graviton) são mais baratas por vCPU, o que reduz a linha de computação, a validar com os testes de carga.

6.3 Como a infra garante 99,9% (RNF03)

Componente do caminho crítico	O que o torna redundante	SLA publicado pela AWS (confirmar nas páginas de SLA)
Application Load Balancer	Opera em 2 AZ	99,99%
ECS Fargate	Tarefas em 2 AZ com reposição automática	99,99%
RDS PostgreSQL Multi-AZ	Standby síncrono com failover automático (tipicamente 1 a 2 minutos)	99,95%

Disponibilidade composta em série: 0,9999 × 0,9999 × 0,9995 ≈ 0,9993 (**99,93%**), o que equivale a ≈ 30,7 minutos de indisponibilidade por mês, abaixo do limite de 43,8 minutos de 99,9%. Redis e SQS ficam fora do caminho crítico: se o Redis cair, a API lê do banco (mais lento, porém no ar); se a fila falhar, só e-mails atrasam. Observação: o cálculo usa SLAs contratuais, não incidentes reais, e não cobre falha de software nosso, por isso existem o rollback automático (6.4) e os alertas (6.7).

6.4 Pipeline CI/CD (GitHub Actions)

#	Etapa	O que faz	Critério de bloqueio
1	Lint	Checkstyle (Java), ESLint e Prettier (Next.js)	Qualquer erro
2	Testes unitários	JUnit 5 e Jest	Falha de teste ou cobertura abaixo de 80% no domínio / 70% global (RNF09)
3	Build	Maven, next build e docker build multi-stage	Falha de build
4	Testes de integração	Testcontainers com PostgreSQL e Redis reais	Falha de teste
5	Segurança	CodeQL (SAST), dependências (Dependabot/OWASP Dependency-Check), imagem (Trivy), segredos (gitleaks)	Qualquer vulnerabilidade crítica ou alta (RNF05)
6	Publicação	Envia a imagem ao ECR com tag igual ao hash do commit	Falha de push
7	Deploy em staging	Terraform plan/apply, migrations Flyway, atualização do serviço ECS	Falha de migration ou de deploy
8	Smoke test	Health check, login, listagem de planos e abertura de simulado	Qualquer falha dispara rollback automático
9	Aprovação	Revisão humana (PO ou líder técnico)	Sem aprovação não vai à produção
10	Deploy em produção	Rolling update com circuit breaker do ECS e novo smoke test	Falha reverte para a versão anterior

Testes ponta a ponta com Playwright rodam toda noite em staging (não bloqueiam o deploy do dia, mas abrem alerta). **Por que GitHub Actions e não Jenkins:** não exige servidor próprio para manter, o que combina com a equipe pequena (ADR 3.2).

6.5 Containerização

Dockerfile da API (multi-stage): o primeiro estágio compila com JDK e Maven; o segundo contém só o JRE e o jar, o que reduz o tamanho e a superfície de ataque.

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -q dependency:go-offline
COPY src ./src
RUN mvn -q package -DskipTests

FROM eclipse-temurin:21-jre-alpine
RUN addgroup -S app && adduser -S app -G app
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
USER app
EXPOSE 8080
ENTRYPOINT exec java -jar app.jar
```

Dockerfile do Web (multi-stage), com `output: 'standalone'` no `next.config.js`:

```
FROM node:22-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci

FROM node:22-alpine AS build
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build

FROM node:22-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY --from=build /app/.next/standalone ./
COPY --from=build /app/.next/static ./.next/static
COPY --from=build /app/public ./public
USER node
EXPOSE 3000
CMD node server.js
```

docker-compose.yml (desenvolvimento local)

```
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: trilha
      POSTGRES_USER: trilha
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    ports: ['5432:5432']
    volumes: ['dbdata:/var/lib/postgresql/data']
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U trilha']
      interval: 5s
      retries: 10
  redis:
    image: redis:7-alpine
    ports: ['6379:6379']
  api:
    build: ./backend
    env_file: .env
    ports: ['8080:8080']
    depends_on:
```

```
            db:
              condition: service_healthy
            redis:
              condition: service_started
        web:
          build: ./frontend
          env_file: .env
          ports: ['3000:3000']
          depends_on: [api]
      volumes:
        dbdata:
```

6.6 Segurança

- **Autenticação:** JWT de acesso (15 min) e refresh rotativo (7 dias) com revogação em Redis (ADR-010). Administradores e financeiro usam verificação em duas etapas. OAuth2/OIDC (login social) fica fora do MVP e no backlog.
- **Autorização:** RBAC para o perfil (RF005) mais verificação de propriedade do recurso, que é ABAC simples: um aluno só lê as próprias tentativas, redações e pagamentos; um professor só corrige redações atribuídas a ele.
- **Segredos:** AWS Secrets Manager, com rotação de credenciais do banco. Descartados: arquivo `.env` no repositório e variáveis fixas na imagem (vazam no histórico do Git e nas camadas Docker).

OWASP Top 10:2021 endereçado (conferir se já existe edição mais recente antes de entregar):

Item	Risco	Controle no projeto	Onde se verifica
A01	Quebra de controle de acesso	RBAC + checagem de propriedade; negar por padrão	Testes de integração por perfil
A02	Falhas criptográficas	TLS 1.2+, criptografia em repouso (KMS), CPF cifrado em coluna, bcrypt	Revisão de arquitetura
A03	Injeção	JPA com consultas parametrizadas; validação de entrada	CodeQL no pipeline
A04	Design inseguro	Modelagem de ameaças por módulo; limite de redações e de taxa	Revisão por sprint
A05	Configuração incorreta	Sub-redes privadas, security groups mínimos, Terraform versionado	Terraform plan revisado
A06	Componentes vulneráveis	Dependabot, Dependency-Check e Trivy	Etapa 5 do pipeline
A07	Falhas de autenticação	Limite de 5 tentativas por minuto, bcrypt, token curto, mensagem genérica	Sequência 1 (4.3)
A08	Falhas de integridade	Assinatura HMAC do webhook, imagens por hash de commit	Sequência 3 (4.3)
A09	Falhas de log e monitoramento	Logs estruturados, auditoria, alertas	6.7
A10	SSRF	A API não busca URLs informadas pelo usuário; saída restrita por security group	Revisão de código

6.7 Observabilidade

- **Logs:** JSON no CloudWatch Logs, com `correlation_id` em 100% das requisições (RNF10); nunca registrar senha, token ou CPF.
- **Métricas técnicas:** latência p50/p95/p99, taxa de erro 5xx, CPU e memória das tarefas, conexões e CPU do banco, tamanho da fila SQS.
- **Métricas de negócio:** cadastros por dia, conversão para assinante, simulados concluídos, redações na fila e idade da mais antiga, pagamentos confirmados e estornos.

Alerta	Condição	Ação
Latência alta	p95 acima de 300 ms por 5 min	Notifica o plantão; avalia escalar
Erros	5xx acima de 1% por 5 min	Notifica; avalia rollback
Banco	CPU acima de 70% por 15 min ou créditos zerados	Aciona o gatilho de escala (6.2)
Fila de redação	Redação sem correção há mais de 48 h	Avisa o administrador (RNF11)
Webhook	Falhas de assinatura ou 5xx no webhook	Notifica financeiro e plantão
Custo	Gasto mensal acima de 80% do orçamento definido	Notifica o líder técnico (R05)

Bloco 7. Planejamento e Estimativas

Regra deste bloco: custo inventado ou equipe sem pesquisa salarial real zera o bloco. Os salários abaixo vêm de fontes públicas citadas e as lacunas estão marcadas.

7.1 WBS (Work Breakdown Structure, 4 níveis)

- **1. Trilha Enem (MVP)**
  - **1.1 Gestão do projeto**
    - 1.1.1 Planejamento e backlog (1.1.1.1 refinamento das histórias, 1.1.1.2 estimativa e priorização)
    - 1.1.2 Cerimônias Scrum (planning, daily, review, retrospectiva)
    - 1.1.3 Gestão de riscos e de custos (matriz de riscos, alertas de custo)
  - **1.2 Fundação técnica**
    - 1.2.1 Repositórios e padrões (1.2.1.1 fluxo Git, 1.2.1.2 guia de código, 1.2.1.3 Checkstyle/ESLint)
    - 1.2.2 Infraestrutura como código (1.2.2.1 rede VPC, 1.2.2.2 ECS e ALB, 1.2.2.3 RDS e Redis, 1.2.2.4 S3 e CloudFront)
    - 1.2.3 CI/CD e observabilidade (1.2.3.1 pipeline de 10 etapas, 1.2.3.2 logs, métricas e alertas)
  - **1.3 Identidade e conta (F1)**
    - 1.3.1 Cadastro e consentimento do responsável (RF001, RF002)
    - 1.3.2 Login, recuperação de senha e perfis (RF003 a RF005)
    - 1.3.3 Exportação e exclusão de dados (RF006)
  - **1.4 Assinatura e pagamentos (F2, F10)**
    - 1.4.1 Planos e assinatura (RF007, RF010)
    - 1.4.2 PIX e webhook idempotente (RF008, RF012)
    - 1.4.3 Cartão recorrente e recibos (RF009, RF011)
  - **1.5 Conteúdo (F3, F4)**
    - 1.5.1 Aulas, vídeo adaptativo e progresso (RF013 a RF015)
    - 1.5.2 Banco de questões (RF016)
  - **1.6 Avaliação (F5, F6)**
    - 1.6.1 Geração e realização de simulado (RF017, RF018)
    - 1.6.2 Correção automática e relatório (RF019, RF020)
  - **1.7 Redação (F7)**
    - 1.7.1 Envio e fila de correção (RF021, RF023)
    - 1.7.2 Correção por competência (RF022)
  - **1.8 Plano de estudos e mentoria (F8, F9)**
    - 1.8.1 Geração do plano (RF024)
    - 1.8.2 Agenda de mentoria (RF025)
  - **1.9 Gestão e suporte (F11, F12)**
    - 1.9.1 Administração de usuários, turmas e planos (RF027)
    - 1.9.2 Chamados, notificações e painel de gestão (RF026, RF028 a RF030)
  - **1.10 Qualidade e lançamento**
    - 1.10.1 Testes de carga com 10.000 usuários (RNF02)
    - 1.10.2 Revisão de segurança e LGPD (RNF05, RNF06)
    - 1.10.3 Documentação, treinamento de professores e go-live

7.2 Estimativa de esforço

Velocidade da equipe: 40 pontos por sprint [P]. Base: 5 pessoas produtivas × 8 pontos cada por sprint de 2 semanas. É uma hipótese e deve ser recalibrada ao fim das sprints 1 e 2 com a velocidade real.

Funcionalidade (WBS)	Story points	Por que esse tamanho
Infra, CI/CD e observabilidade (1.2)	13	Terraform, pipeline e alertas desde o início
F1 Identidade e conta	21	Consentimento de menores e LGPD aumentam a complexidade
F2 Assinatura	13	Regras de estado simples
F3 Aulas	21	Vídeo adaptativo com URL assinada
F4 Banco de questões	21	CRUD com validações e publicação
F5 Simulados	34	Maior risco técnico: lote, idempotência e carga
F6 Desempenho	13	View materializada + telas
F7 Redação	34	Fila, atribuição, rubrica e prazo
F8 Plano de estudos	21	Algoritmo de geração (Strategy)
F9 Mentoria	13	Agenda sem sobreposição (constraint no banco)
F10 Pagamentos	34	Gateway, webhook, estorno e conciliação
F11 Gestão	21	CRUDs administrativos com RBAC
F12 Suporte e painel	21	Chamados com SLA e dashboards
Qualidade, LGPD e testes de carga (1.10)	13	Reserva para endurecimento

Total	**293**	

Cronograma em 8 sprints de 2 semanas (16 semanas):

Sprint	Objetivo	Pontos
1	Infra, CI/CD e F1 (cadastro, consentimento, login)	34
2	F2 planos e assinatura + F3 aulas	34
3	F4 banco de questões + início de F10 (PIX e webhook)	40
4	Fim de F10 + F5 geração e realização de simulado	40
5	Fim de F5 + F6 desempenho + início do plano de estudos	40
6	Fim do plano de estudos + F7 redação	37
7	F9 mentoria + F11 gestão	34
8	F12 suporte e painel + qualidade, LGPD e testes de carga	34
Total		**293**

Por que 8 sprints: 293 pontos ÷ 40 = 7,3 sprints, arredondado para 8. A capacidade total é 320 pontos, então há folga de 27 pontos (≈ 9% sobre o escopo) para imprevistos. Com 6 sprints seria preciso uma velocidade de 49 pontos (22% acima da hipótese, irreal para uma equipe nova); 10 sprints encareceriam o projeto em 2 meses de equipe sem necessidade. Iniciando em dezembro de 2026, o MVP fica pronto em abril de 2027, com meses de folga antes do Enem 2027 (datas oficiais a confirmar no edital do Inep).

7.3 Composição da equipe e custo

Fontes salariais consultadas em 07/11/2026: médias do Indeed Brasil (o enunciado cita Glassdoor: confirmar cada valor no Glassdoor e anexar o print; se divergir, atualizar a tabela). Desenvolvedor Java pleno: R$ 6.340 a R$ 6.422 por mês (Brasil/remoto); desenvolvedor Java sênior: R$ 10.532 por mês [F]. Para o líder técnico foi adotado o valor sênior. A faixa de arquiteto de software no mesmo site (R$ 15.876) vem de uma amostra de apenas 5 salários e não foi usada.

Perfil	Qtd.	Salário mensal	Origem do valor	Custo mensal com encargos (× 1,8)
Líder técnico / arquiteto (Java sênior)	1	R$ 10.532	[F] Indeed, Java sênior	R$ 18.957,60
Desenvolvedor back-end (Java pleno)	2	R$ 6.340	[F] Indeed, Java pleno	R$ 22.824,00
Desenvolvedor front-end (Next.js pleno)	1	R$ 6.340	[P] igual ao pleno: sem dado verificado, [COTAR] Glassdoor	R$ 11.412,00
QA / automação de testes (pleno)	1	R$ 6.340	[P] igual ao pleno: [COTAR] Glassdoor	R$ 11.412,00
PO / analista (50% de dedicação)	0,5	R$ 6.340 (valor integral)	[P] [COTAR] Glassdoor	R$ 5.706,00
Total mensal		**R$ 39.062**		**R$ 70.311,60**

O fator 1,8 sobre o salário (encargos, férias, 13º, benefícios) é uma premissa [P] comum em planejamento de custo CLT; o fator real depende do regime de contratação e deve ser validado com a contabilidade.

Custo total do desenvolvimento (8 sprints = 4 meses): R$ 70.311,60 × 4 = **R$ 281.246,40**. A isso soma-se a infraestrutura de staging durante o desenvolvimento: US$ 161,91 × 4 = US$ 647,64, mais as linhas a cotar (6.2), convertidos na cotação do dia [COTAR].

Por que esse perfil de equipe:

- **Líder técnico sênior:** decisões de arquitetura, revisão de código e Terraform. Reduz o risco R06 de atraso.
- **2 desenvolvedores back-end:** Avaliação (34 pontos) e Pagamentos (34 pontos) concentram 23% do esforço e o maior risco técnico, e merecem duas pessoas.
- **1 desenvolvedor front-end:** o front é compartilhado por 4 perfis; risco de gargalo mitigado com componentes reutilizáveis e apoio do líder técnico.
- **1 QA:** automação de testes é pré-requisito da definição de pronto (Bloco 9) e dos testes de carga.
- **Sem DevOps dedicado:** Fargate e Terraform dispensam operação de nós (ADR-006). Contratar um DevOps seria custo de ≈ R$ 11 mil por mês sem demanda que o justifique.

Viabilidade econômica (com as premissas P01 a P04):

Cenário	Pagantes	Receita bruta mensal	Leitura
Meta (P02)	2.000	R$ 119.800	Cobre folgadamente a equipe de manutenção e a infraestrutura
Pessimista (5%)	1.000	R$ 59.900	Ainda viável; exige conter custo de vídeo e de correção

Para recuperar só o desenvolvimento em 12 meses: R$ 281.246,40 ÷ (R$ 59,90 × 12) ≈ 392 assinantes pagando o ano inteiro, sem contar infra, correção de redações (400 h/mês, valor por hora a definir) e taxas do gateway (a validar). Isso indica viabilidade, desde que a conversão não fique muito abaixo da meta.

7.4 Matriz de riscos (11 riscos)

Escala: P (probabilidade) e I (impacto) de 1 a 5; severidade = P × I. Faixas: 1 a 6 baixa, 7 a 12 média, 13 ou mais alta.

ID	Categoria	Risco	P	I	Sev.	Mitigação	Gatilho de acompanhamento
R03	Negócio	Conversão abaixo de 10%	4	4	16	Plano gratuito limitado, testes de preço, cenário de 5% no financeiro	Conversão menor que 7% após 60 dias
R01	Técnico	Pico de acesso derruba o simulado	3	5	15	Autoscaling, cache, lote de 30 s, teste de carga antes do Enem	p95 acima de 300 ms nos testes de carga
R05	Negócio/Técnico	Custo de transferência de vídeo acima do previsto	3	4	12	Vídeo adaptativo, métricas de consumo, alerta de custo	Gasto de CDN acima de 80% do orçamento
R04	Equipe	Falta de professores corretores	3	4	12	Limite de redações por plano, fila com prazo, recrutar além dos 5	Fila com mais de 48 h
R11	Técnico	Banco burstable esgota créditos de CPU no pico	3	4	12	Teste de carga, gatilho de migração de instância (6.2)	Créditos zerados

R02	Segurança	Vazamento de dados de menores	2	5	10	Criptografia, RBAC, auditoria, PII mínima	Falha crítica no pipeline de segurança
R10	Regulatório	Não conformidade com a LGPD	2	5	10	Consentimento versionado, revisão jurídica do termo, prazo de 15 dias ao titular	Solicitação de titular sem resposta em 10 dias
R06	Equipe/Prazo	Atraso no desenvolvimento	3	3	9	Escopo Must primeiro, folga de 27 pontos, replanejar por velocidade real	Velocidade menor que 32 pontos por 2 sprints
R08	Técnico/Negócio	Gateway indisponível ou com taxas alteradas	2	4	8	Camada Adapter, plano B de gateway, reprocesso do webhook	Taxa de falha no webhook acima de 2%
R09	Equipe	Saída de membro-chave	2	4	8	Revisão de código em pares, documentação (Bloco 8), conhecimento compartilhado	Aviso de desligamento
R07	Regulatório	Mudança de matriz ou datas do Enem pelo Inep	2	3	6	Conteúdo atualizável sem deploy, monitorar editais	Publicação de novo edital

Os riscos R01 a R06 vêm do Bloco 1 (1.7). Os riscos de maior severidade (R03, R01) já têm medidas no desenho: o plano gratuito e o cenário pessimista no financeiro para R03, e a arquitetura de carga para R01.

Bloco 8. Documentação para o Dev

Critério de sucesso: o desenvolvedor chega no dia 1, clona o repositório e sobe o sistema sem fazer pergunta. Tudo abaixo é o conteúdo inicial do repositório `trilhaenem` (endereço do Git a definir).

8.1 README completo (do clone ao localhost)

```
TRILHA ENEM
Plataforma web de preparacao para o Enem: aulas, simulados, correcao de redacao e plano de estudos.

PRE-REQUISITOS
- Git
- Docker 24 ou superior com Docker Compose
- JDK 21 e Node.js 22 (so para rodar fora do Docker)

COMO RODAR EM 5 PASSOS
1. Clonar:         git clone <url-do-repositorio> && cd trilhaenem
2. Configurar:     cp .env.example .env   (trocar DB_PASSWORD, JWT_SECRET e CPF_ENCRYPTION_KEY)
3. Subir tudo:     docker compose up --build
4. Abrir:          web http://localhost:3000   API http://localhost:8080
                   documentacao da API http://localhost:8080/swagger-ui.html
                   saude http://localhost:8080/actuator/health
5. Usuarios de teste (criados pela migration de desenvolvimento):
                   aluno@teste.local, professor@teste.local, admin@teste.local (senha no .env.example)

RODAR SO A INFRA E EXECUTAR O CODIGO NA MAQUINA
- docker compose up -d db redis
- backend:  cd backend && ./mvnw spring-boot:run
- frontend: cd frontend && npm ci && npm run dev

TESTES
- Backend:   cd backend && ./mvnw verify      (unitarios e integracao com Testcontainers)
- Frontend:  cd frontend && npm test
- Carga:     k6 run tests/load/simulado.js     (meta: p95 abaixo de 300 ms)

ESTRUTURA DE PASTAS
backend/   Java 21 e Spring Boot. Um pacote por modulo: identidade, assinatura, conteudo,
           avaliacao, redacao, planoestudo, suporte, relatorios, notificacoes.
           Migrations Flyway em src/main/resources/db/migration
frontend/  Next.js e TypeScript (app/, components/, lib/)
infra/     Terraform (rede, ECS, RDS, Redis, S3 e CloudFront)
docs/      adr/ (decisoes), diagramas/ (fontes PlantUML), openapi.yaml
.github/   workflows do pipeline

DOCUMENTOS OBRIGATORIOS ANTES DE CONTRIBUIR
docs/adr/, guia de codigo (secao 8.4) e Git workflow (secao 8.5).
```

8.2 Variáveis de ambiente (.env.example)

```
# Banco de dados
DB_HOST=localhost
DB_PORT=5432
DB_NAME=trilha
DB_USER=trilha
DB_PASSWORD=troque-esta-senha

# Redis
REDIS_HOST=localhost
REDIS_PORT=6379

# Seguranca (gerar com openssl rand -base64 48)
JWT_SECRET=gere-um-segredo-longo
JWT_ACCESS_TTL_MINUTES=15
JWT_REFRESH_TTL_DAYS=7
CPF_ENCRYPTION_KEY=gere-com-openssl-rand-base64-32

# Pagamento (usar credenciais de sandbox em desenvolvimento)
PAYMENT_GATEWAY_BASE_URL=https://api.mercadopago.com
PAYMENT_ACCESS_TOKEN=token-de-sandbox
PAYMENT_WEBHOOK_SECRET=segredo-do-webhook

# AWS (em producao os segredos vem do Secrets Manager; localmente use um emulador opcional como LocalStack)
AWS_REGION=sa-east-1
SQS_QUEUE_URL=http://localhost:4566/000000000000/trilha-eventos
SES_FROM_ADDRESS=nao-responda@trilhaenem.com.br
VIDEO_CDN_BASE_URL=https://cdn.exemplo.local
VIDEO_URL_SIGNING_KEY=gere-uma-chave

# Frontend
NEXT_PUBLIC_API_URL=http://localhost:8080/api/v1
```

Regra: `.env` nunca vai para o Git (está no `.gitignore`); só o `.env.example`, sem valores reais.

8.3 Contrato de API (OpenAPI 3.0, trecho inicial)

Cobre autenticação (Sprint 1), simulado (Sprints 3 e 4) e webhook de pagamento. O arquivo completo fica em `docs/openapi.yaml`; o trecho abaixo teve a sintaxe YAML validada e acompanha a entrega (`openapi.yaml`).

```yaml
openapi: 3.0.3
info:
  title: Trilha Enem API
  version: 1.0.0
  description: Contrato inicial. Autores Jose Felipe, Arionaldo Junior e Lucas Leal.
servers:
  - url: http://localhost:8080/api/v1
paths:
  /auth/login:
    post:
      summary: Autentica o usuario
      requestBody:
        required: true
```

```yaml
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/LoginRequest'
      responses:
        '200':
          description: Tokens emitidos
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/TokenResponse'
        '401':
          description: Credenciais invalidas
        '403':
          description: Consentimento do responsavel pendente
        '429':
          description: Limite de tentativas excedido
  /simulados:
    post:
      summary: Gera um simulado e inicia uma tentativa
      security:
        - bearerAuth: []
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/SimuladoRequest'
      responses:
        '201':
          description: Tentativa criada com as questoes
        '403':
          description: Assinatura inativa
  /tentativas/{id}/respostas:
    put:
      summary: Salva um lote de respostas (idempotente)
      security:
        - bearerAuth: []
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: string
            format: uuid
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/Resposta'
      responses:
        '204':
          description: Lote salvo
        '404':
          description: Tentativa nao encontrada ou de outro aluno
  /webhooks/pagamentos:
    post:
      summary: Recebe a confirmacao do gateway (assinatura HMAC no cabecalho)
      responses:
        '200':
          description: Processado ou ja processado antes
        '401':
          description: Assinatura invalida
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
  schemas:
    LoginRequest:
      type: object
      required: [email, senha]
      properties:
        email:
          type: string
          format: email
        senha:
          type: string
          minLength: 8
    TokenResponse:
      type: object
      properties:
        accessToken:
          type: string
        refreshToken:
          type: string
        expiraEmSegundos:
          type: integer
    SimuladoRequest:
      type: object
      required: [areaId, duracaoMin]
      properties:
        areaId:
          type: integer
        duracaoMin:
          type: integer
          minimum: 10
        dificuldade:
          type: integer
          minimum: 1
          maximum: 5
    Resposta:
      type: object
      required: [questaoId, alternativa]
      properties:
        questaoId:
          type: string
          format: uuid
        alternativa:
          type: string
          enum: [A, B, C, D, E]
```

8.4 Guia de padrões de código

Tema	Regra	Por quê
Idioma	Domínio em português (Aluno, Assinatura, Redacao); termos técnicos de framework em inglês	O vocabulário do PO, do dicionário de dados e do código fica igual (glossário, 9.1)
Estrutura Java	Um pacote por módulo; módulos só conversam por interfaces públicas e eventos	Mantém a modularidade do monolito (ADR-001)
Camadas	Controller só traduz HTTP; regra de negócio no serviço; acesso a dados no repositório	Testabilidade e troca de adaptadores
DTOs	Java `record` imutáveis na borda; entidades JPA nunca saem na API	Evita vazar campos (PII, senha_hash)
Transações	`@Transactional` nos serviços de caso de uso, nunca no controller	Consistência do webhook e do simulado
Erros	Exceções de domínio convertidas em respostas padronizadas (Problem Details)	Cliente trata erros de forma uniforme
Logs	JSON com correlation_id; proibido registrar senha, token ou CPF	RNF10 e LGPD
Testes	Nome no formato `deve...Quando...`; um comportamento por teste	Documentam a regra
TypeScript	`strict` ligado, sem `any`, componentes funcionais, ESLint e Prettier no CI	Menos erros em tempo de execução
Complexidade	Método com no máximo 10 de complexidade ciclomática	RNF09
Segredos	Nunca no código nem no Git	OWASP A02 e A05

8.5 Git workflow

GitHub Flow com os seguintes acordos:

- `main` é protegida e sempre implantável; ninguém faz push direto.
- Branches curtas (no máximo 3 dias) com o formato `feat/RF018-salvar-lote`, `fix/webhook-duplicado`, `chore/atualiza-dependencias`.
- Commits no padrão Conventional Commits (`feat:`, `fix:`, `docs:`, `test:`, `chore:`).
- Pull request obrigatório, com 1 aprovação (2 em pagamentos e segurança) e pipeline verde. Merge por squash.
- Releases marcadas com tag `vMAIOR.MENOR.PATCH`; correção urgente sai de uma branch a partir de `main`.

Por que não GitFlow: o deploy é contínuo, e as branches `develop` e `release` acrescentam sincronização e conflitos sem benefício para 5 pessoas. **Por que não trabalhar direto na `main`:** com o risco R09 (saída de um membro) e dinheiro envolvido, a revisão de código é obrigatória.

8.6 Sprint 1 detalhado (34 pontos)

Meta da sprint: ambiente funcionando de ponta a ponta (do commit ao staging) e o aluno conseguir se cadastrar, consentir e entrar.

ID	Item	Pontos	Critérios de aceite
S1-01	Estrutura do repositório e padrões	3	*Given* um computador limpo, *when* seguir o README, *then* `docker compose up` sobe web, API, banco e Redis, e o README foi validado por alguém que não o escreveu
S1-02	Terraform base em staging (VPC, ECS, RDS)	5	*Given* credenciais AWS, *when* `terraform apply`, *then* o staging responde em HTTPS e o banco só aceita tráfego das tarefas
S1-03	Pipeline de CI (lint, testes, build, segurança)	5	*Given* um PR com erro de lint ou vulnerabilidade crítica, *then* o merge é bloqueado e o relatório de cobertura é publicado
S1-04	US01 cadastro com consentimento (RF001, RF002)	8	*Given* aluno de 17 anos, *when* envia o cadastro, *then* a conta fica inativa e o responsável recebe um link válido por 7 dias; *given* o aceite, *then* a conta é ativada e o aceite é registrado com data, versão do termo e IP
S1-05	US02 login seguro (RF003)	3	*Given* 5 tentativas erradas em 1 minuto, *then* a sexta recebe 429; *given* credenciais corretas, *then* recebe o token de 15 minutos e o refresh de 7 dias
S1-06	Recuperação de senha (RF004)	3	*Given* e-mail cadastrado, *when* solicita, *then* recebe link de uso único que expira; *given* e-mail desconhecido, *then* a resposta é idêntica (não revela existência)
S1-07	Perfis e permissões (RF005)	4	*Given* um aluno, *when* chama um endpoint de administrador, *then* recebe 403 e o acesso fica na auditoria
S1-08	Exportar e excluir dados (RF006)	3	*Given* solicitação confirmada, *when* processada, *then* o JSON de exportação é entregue e os dados pessoais são anonimizados, mantendo pagamentos
	Total	**34**	Cada item só é concluído atendendo à Definição de Pronto (9.5)

Bloco 9. Qualidade e Processo

9.1 Modelo de domínio e glossário

O modelo de domínio é o diagrama de classes do item 4.2 (15 classes) e o ER do item 5.2. O glossário garante que PO, arquiteto, DBA e devs usem o mesmo vocabulário.

Termo	Definição no projeto
Aluno	Usuário que estuda e assina um plano; pode ser menor de 18 anos
Responsável legal	Pessoa que autoriza o cadastro de um aluno menor e, em geral, paga
Consentimento	Aceite registrado do responsável, com data, versão do termo e IP
Plano / Assinatura	Plano é o produto (Essencial ou Completo); assinatura é o vínculo de um aluno com um plano em um período
Pagamento	Cobrança de uma assinatura (PIX ou cartão), confirmada por webhook do gateway
Webhook	Chamada do gateway ao sistema avisando que um pagamento mudou de estado
Idempotência	Repetir a mesma operação produz o mesmo resultado, sem duplicar efeito (webhook e lote de respostas)
Área de conhecimento	Linguagens, Ciências Humanas, Ciências da Natureza, Matemática (e Redação como área própria)
Questão	Pergunta de múltipla escolha com 5 alternativas, gabarito, área e dificuldade de 1 a 5
Simulado	Conjunto de questões com tempo definido, montado por área e dificuldade
Tentativa	Execução de um simulado por um aluno, com respostas e nota estimada
Nota estimada	Nota calculada por acertos ponderados pela dificuldade. Não é a nota TRI do Enem
Competência	Cada um dos 5 critérios da redação do Enem, com nota de 0 a 200
Plano de estudos (a “trilha”)	Cronograma semanal até a data da prova, gerado pelo desempenho do aluno
Mentoria	Encontro agendado entre aluno e professor
Chamado / SLA	Pedido de suporte; SLA é o prazo de primeira resposta (24 h)
PII	Dado pessoal identificável, tratado conforme a LGPD
MoSCoW	Priorização Must, Should, Could, Won’t
Story point	Medida relativa de esforço de uma história, em escala de Fibonacci

9.2 Estratégia de testes

Nível	Fatia dos testes	Ferramenta	O que cobre	Quando roda
Unitário	≈ 70%	JUnit 5, Mockito, Jest	Regras de domínio: cálculo de nota, estados de assinatura e redação, regra de idade	A cada commit (etapa 2 do pipeline)
Integração	≈ 20%	Spring Boot Test + Testcontainers (PostgreSQL e Redis reais)	Repositórios, constraints do banco (assinatura ativa única, mentoria sem sobreposição), segurança por perfil, webhook idempotente	A cada PR (etapa 4)
Ponta a ponta (E2E)	≈ 10%	Playwright	Jornadas: cadastro com consentimento, assinar por PIX (sandbox), fazer simulado, enviar redação	Toda noite em staging e antes de cada release
Contrato	—	Validação do OpenAPI	A API respeita o openapi.yaml	A cada PR
Carga	—	k6	10.000 usuários simultâneos por 2 h no cenário de simulado; critério: p95 ≤ 300 ms e erros ≤ 1% (RNF01, RNF02)	Sprint 8 e antes de cada Enem
Segurança	—	CodeQL, Trivy, OWASP ZAP (varredura de linha de base)	Vulnerabilidades de código, dependências, imagem e aplicação em execução	Pipeline e semanal em staging

Por que essa proporção (pirâmide): testes unitários são rápidos e baratos e pegam a maior parte das regras; os de integração validam o que só o banco real garante (constraints); poucos E2E, porque são lentos e frágeis. Os critérios de aceite Given/When/Then do Bloco 2 viram os cenários dos testes de integração e E2E, o que liga requisitos e testes.

9.3 Metas de cobertura

Escopo	Meta	Ferramenta
Camada de domínio e serviços	≥ 80% de linhas	JaCoCo
Projeto inteiro (backend)	≥ 70% de linhas	JaCoCo
Módulos Avaliação e Pagamentos (caminho crítico)	≥ 75% de ramos	JaCoCo
Frontend	≥ 70% das linhas dos componentes de lógica	Jest

O pipeline bloqueia o merge abaixo dessas metas (item 6.4, etapa 2). Cobertura mede o que foi executado, e não a qualidade do teste; por isso a revisão de PR confere se os asserts verificam comportamento.

9.4 Definição de Pronto (DoR, Definition of Ready)

Uma história só entra na sprint se:

- [ ] Segue o formato “Como [persona], quero [ação], para [objetivo]”.
- [ ] Tem critérios de aceite Given/When/Then.
- [ ] Está estimada em story points (até 13; acima disso, dividir).

- [ ] Tem requisitos (RF e RNF) e protótipo ou rascunho de tela referenciados.
- [ ] Dependências identificadas (por exemplo, o contrato da API já está no OpenAPI).
- [ ] O PO e pelo menos um desenvolvedor concordam que ela é compreensível.

9.5 Definição de Feito (DoD, Definition of Done)

Uma história só é concluída se:

- [ ] Código revisado e aprovado em pull request e mesclado em `main`.
- [ ] Pipeline verde: lint, testes, cobertura nas metas (9.3) e varredura de segurança sem vulnerabilidade crítica ou alta.
- [ ] Testes automatizados cobrem os critérios de aceite.
- [ ] Migration versionada, aplicada em staging e compatível com a versão anterior da aplicação.
- [ ] OpenAPI atualizado quando a API muda.
- [ ] Campos PII novos marcados no dicionário de dados (RNF06).
- [ ] Logs e métricas necessários criados; nenhum dado sensível no log.
- [ ] Implantada em staging com smoke test passando.
- [ ] O PO validou o critério de aceite em staging.

9.6 Metodologia: Scrum, com Kanban na operação

Aspecto	Decisão	Alternativa descartada	Motivo
Construção do MVP	Scrum com sprints de 2 semanas	Kanban puro	Há um marco com data (pronto antes do pico do Enem) e a estimativa do Bloco 7 é em sprints; Scrum dá previsibilidade, revisão e retrospectiva a cada 2 semanas
Papéis	PO: José Felipe. Líder técnico acumula a facilitação das cerimônias. Time: devs e QA	Scrum Master dedicado	Equipe de 5 pessoas não justifica o custo de um papel exclusivo
Cerimônias	Planning (2 h), daily (15 min), review (1 h), retrospectiva (1 h)	—	Mantém inspeção e adaptação sem excesso de reuniões
Operação após o go-live	Kanban para bugs e suporte	Continuar só com Scrum	Chamados e correções chegam de forma contínua e não cabem em sprints fechadas; limite de trabalho em andamento evita acúmulo

Coerência entre os blocos

A regra do enunciado: inconsistência entre blocos perde nota. Verificação feita:

Se definimos...	Então obrigatoriamente...	Onde está atendido
RNF03: 99,9% de disponibilidade	Load balancer, redundância e failover	ALB em 2 AZ, tarefas em 2 AZ e RDS Multi-AZ (6.3). Custo Multi-AZ incluído (6.2)
Monolito modular (ADR-001)	O banco é único, sem banco por serviço	PostgreSQL único (5.1). Nenhum diagrama mostra banco por serviço (4.4, 4.5)
PostgreSQL (ADR-004)	FKs, constraints e normalização correta	DDL com PK, FK, CHECK, UNIQUE parcial e EXCLUDE; 3FN justificada (5.3 e 5.4)
LGPD nos RNFs (RNF06)	Dicionário marca campos com tratamento definido	Coluna PII/tratamento (5.5), exclusão por anonimização e retenção (5.6)
8 sprints no planejamento	Estimativa cabe na equipe definida	293 pontos para 320 de capacidade com 5 pessoas e velocidade de 40 (7.2 e 7.3)
RNF12: lote de 30 s	Cliente, API e banco suportam o lote idempotente	RF018, sequência 2 (4.3), PK composta de respostas (5.4) e OpenAPI (8.3)
Next.js e Spring Boot como contêineres separados	Dois serviços ECS e dois Dockerfiles	C4 nível 2 (3.4), custo (6.2) e containerização (6.5)
Custo de pessoal e de infra reais	Fontes citadas e lacunas visíveis	Marcas [F], [P] e [COTAR] em todo o documento

Checagem contra a lista “o que zera o bloco”: arquitetura com ADR (3.1, ok); classes com tipos e visibilidade (4.2, ok); banco com tabela de comparação (5.1, ok); RNFs com métrica (2.2, ok); custo real: depende das linhas [COTAR] ainda não preenchidas; inconsistência entre blocos (tabela acima, ok); diagrama sem explicação (cada diagrama tem o roteiro “Como explicar”); arguição sem resposta (guia da equipe).